

**Perfect Valley Furniture Company: Database Development, Data Analysis,
Database Optimization**

Group B

rl5721	Rui Liu
jz7774	Jinyang Zhang
kk6146	Kanokkan Kanjanapiban
yh6548	Ya-cheng Hsieh

Content

1. Content of Project Phase 1	1
1.1 Project business case and data models	1
1.1.1 Logical Data Model (Group B original)	2
1.1.1 Relational Data Model (Group B original)	3
1.2 Explanation about relationship among all tables	3
1.2.1 Relationship of GB_COUNTRY and GB_CUST (1:M)	3
1.2.2 Relationship of GB_CUST and GB_ORDER (1:M)	3
1.2.3 Relationship of GB_ORDER and GB_ORD_PROD (1:M)	4
1.2.4 Relationship of GB_PROD and GB_ORD_PROD (1:M)	4
1.2.5 Relationship of GB_FINISH and GB_ORD_PROD (1:M)	4
1.2.6 M:N Relationship Solved by Through GB_ORD_PROD	4
1.3 Insert statements for all tables data insertions from the stage tables	5
1.3.1 Create database for the group project, all business tables and Import all stage tables	5
1.3.2 Insert data into all business tables from the stage tables	8
1.4 Screenshots of queries and corresponding results of all data dictionary queries, and record count queries and their result	16
1.4.1 REM, List of Tables	16
1.4.2 REM, List of Table Columns	16
1.4.3 REM, List of Table Column Constraints	18
1.4.4 REM, List of Table Column Comments	19
1.4.5 Retrieving number records for each table	20
1.4.6 Record Counts of each tables.	20
2. Content of Project Phase 2	22
2.1 Three table joins SQL queries (minimum 3 table joins)	22
2.1.1 Table join 1	22
2.1.2 Table join 2	24
2.1.3 Table join3	31
2.2. Write 3 SQL queries, using subqueries (use minimum 3 tables)	36
2.2.1 Subqueries 1	36
2.2.2 Subqueries 2	37
2.2.3 Subqueries 3	39
2.3. Write 2 SQL queries using analytical functions , one using ROLLUP and other using RANK() functions	40
2.3.1 ROLLUP	40
2.3.2 RANK	50
3. Content of Project Phase 3	52

3.1 Two non-unique index.....	52
3.1.1 cust_id on table gb_order	52
3.1.2 lname on table gb_cust.....	53
3.2 Two composite Indexes	54
3.2.1 Composite Index on gb_ord_prod(order_id, prod_id)	54
3.2.2 Composite Index on gb_cust(country_id, state).....	54
3.3 Two function based index.....	55
3.3.1 Index idx_order_month.....	55
3.3.2 Index idx_lname_upper	56
3.4 List of All Indexes Using SQL.....	56
3.4.1 SQL	56
3.4.2 Screenshot	57
3.5 Screenshot of indexes from MySQL workbench.....	58
4. Database procedure to collect statistics of all tables, and call the procedure..	58
4.1 Procedure code and Call statement.....	58
4.2 Result of following dictionary query for index statistics	61
5. Screenshots of each query, and corresponding execution plan	61
5.1 SQL QUERY 1 (Table Join 1)	61
5.2 SQL QUERY 2 (Table Join 2)	63
5.3 SQL QUERY 3 (Table Join 3)	64
5.4 SQL QUERY 4 (Subqueries 1).....	66
5.5 SQL QUERY 5 (Subqueries 2).....	67
5.6 SQL QUERY 6 (Subqueries 3).....	69
5.7 SQL QUERY 7 (ROLLUP)	70
5.8 SQL QUERY 8 (RANK).....	71
6. Conclusion.....	72
7. Team reflection.....	73
8. Individual reflection	74

1. Content of Project Phase 1

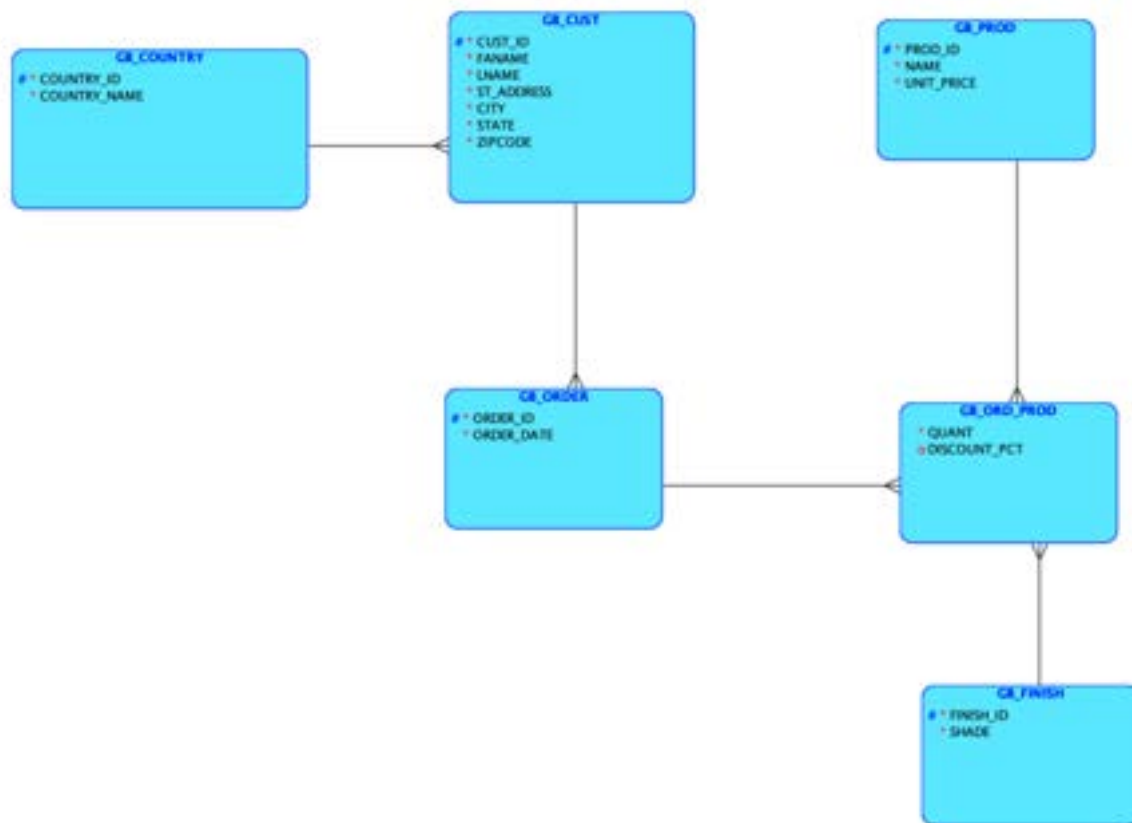
1.1 Project business case and data models

Early computer applications at Perfect Value Furniture Company (PVFC) relied on an old file processing approach. By the late 2000s, competition in the furniture manufacturing industry had intensified, and many competitors were responding more quickly than PVFC to new business opportunities. Although several factors contributed to this trend, managers believed that PVFC's computer information systems based on outdated file-processing methods were a major limitation.

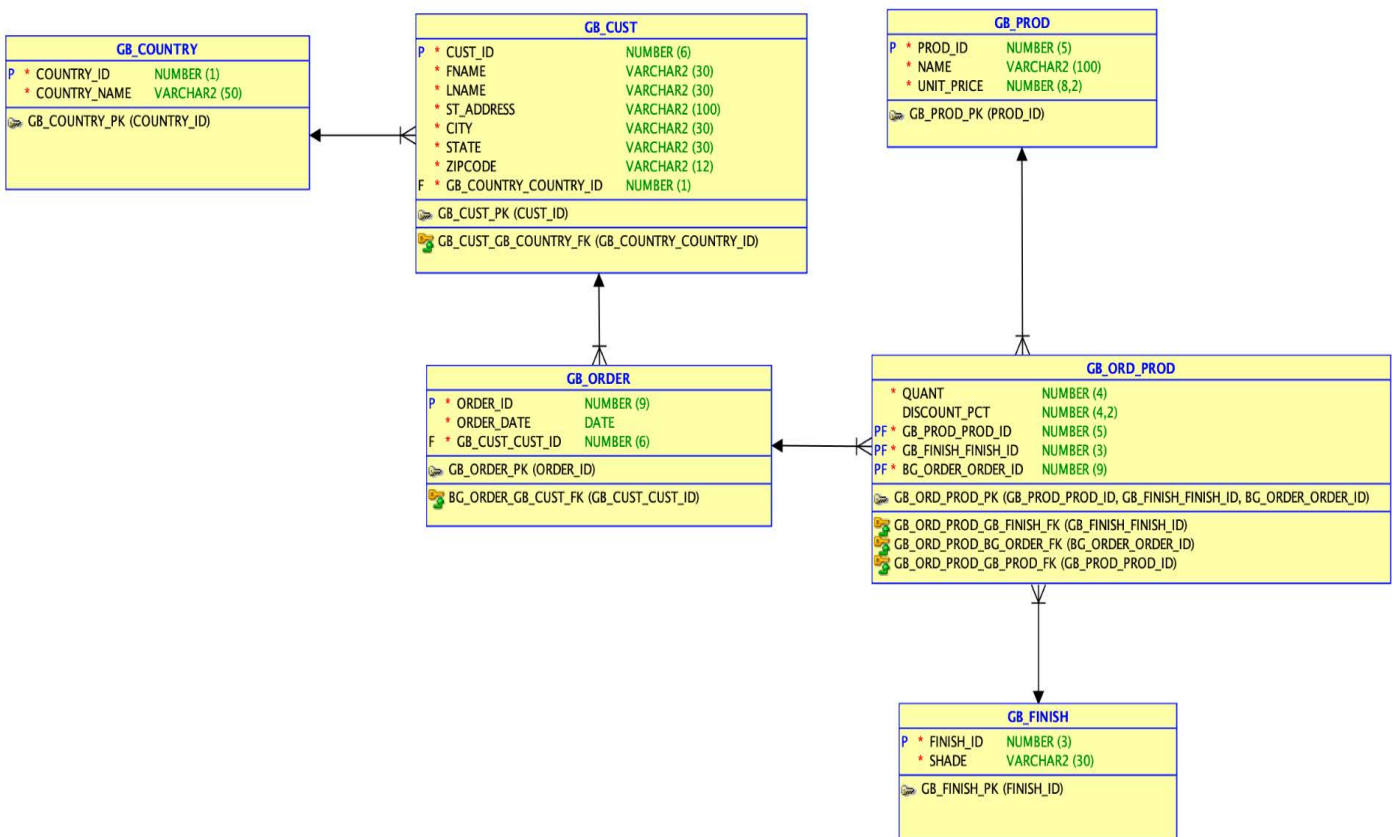
To address this issue, the company launched a development initiative that ultimately led to adopting a modern database approach. Data that had previously been stored in separate files were integrated into a unified database structure, and the metadata describing these data were stored within the same environment. The DBMS now serves as the interface between the organization's various applications and the underlying database(s).

After successfully implementing the transaction-processing database, PVFC plans to expand its database solution to support analytical processing in order to take full advantage of historical transactional data. To avoid placing additional workload on the transactional system, PVFC intends to develop a data warehouse that will capture daily changes from the transactional database and store them for reporting and analysis.

1.1.1 Logical Data Model (Group B original)



1.1.1 Relational Data Model (Group B original)



1.2 Explanation about relationship among all tables

1.2.1 Relationship of GB_COUNTRY and GB_CUST (1:M)

Explain:

- The relationship between the entity GB_COUNTRY and the entity GB_CUST is a one-to-many (1:M) relationship.
- The reason is a country can have many customers, but each customer must be associated with one country only. This is enforced by the foreign key COUNTRY_ID in GB_CUST.
- This relationship is mandatory for GB_CUST because a customer record must have a valid country associated with it.
- For the entity GB_COUNTRY, the relationship is optional because some countries in the database may not have any customers associated with them.

1.2.2 Relationship of GB_CUST and GB_ORDER (1:M)

Explain:

- The relationship of the entity GB_CUST and the entity GB_ORDER is also a one-to-many (1:M) relationship.
- One customer can have many orders, but each order is associated with one and only one customer through the foreign key CUST_ID in GB_ORDER.
- This relationship is mandatory for GB_ORDER because an order must have a customer associated with it.
- The relationship is optional for GB_CUST, because customers could exist before any orders are placed.

1.2.3 Relationship of GB_ORDER and GB_ORD_PROD (1:M)

Explain:

- The relationship of the entity GB_ORDER and the entity GB_ORD_PROD is a one-to-many (1:M) relationship.
- This relationship is mandatory for GB_ORD_PROD, because a line item cannot exist independently of an order. GB_ORDER is an optional entity since an order can be created before line items are entered.

1.2.4 Relationship of GB_PROD and GB_ORD_PROD (1:M)

Explain:

- The relationship of GB_PROD and GB_ORD_PROD is a one-to-many (1:M) relationship.
- A product may exist in many different orders, represented by several records in the table GB_ORD_PROD.
- The relationship is mandatory for GB_ORD_PROD, but optional for GB_PROD, since products can appear in the catalog even if they have not yet been ordered.

1.2.5 Relationship of GB_FINISH and GB_ORD_PROD (1:M)

Explain:

- The relationship of GB_FINISH and GB_ORD_PROD is a one-to-many (1:M) relationship.
- This is a mandatory relationship for the table GB_ORD_PROD, because every product line item must have a finish specified.
- This is an optional relationship for the table GB_FINISH, because a finish type may exist without being used.

1.2.6 M:N Relationship Solved by Through GB_ORD_PROD

Explain:

- One order may contain multiple products.

- One product may appear in multiple orders.
- To properly represent this relationship, the bridge table GB_ORD_PROD is used with a composite primary key consisting of ORDER_ID, PROD_ID, and FINISH_ID.

1.3 Insert statements for all tables data insertions from the stage tables.

1.3.1 Create database for the group project, all business tables and Import all stage tables

create database gb_pvfc;

	Time	Action	Response	Duration / Fetch Time
1	16:30:13	create database gb_pvfc	1 row(s) affected	0.0075 sec

```
CREATE TABLE gb_country (
  country_id TINYINT NOT NULL COMMENT 'Unique Country ID',
  country_name VARCHAR(50) NOT NULL COMMENT 'Name of the country'
);
```

	Time	Action	Response	Duration / Fetch Time
3	16:31:38	CREATE TABLE gb_country (country_id TINYINT NOT NU...	0 row(s) affected	0.082 sec

```
ALTER TABLE gb_country ADD CONSTRAINT gb_country_pk PRIMARY KEY
( country_id );
```

	Time	Action	Response	Duration / Fetch Time
4	16:32:28	ALTER TABLE gb_country ADD CONSTRAINT gb_country_pk...	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.065 sec

```
CREATE TABLE gb_cust (
  cust_id INT NOT NULL COMMENT 'Unique customer id',
  fname VARCHAR(30) NOT NULL COMMENT 'First name of the customer',
  lname VARCHAR(30) NOT NULL COMMENT 'Last name of the customer',
  st_address VARCHAR(100) NOT NULL COMMENT 'Street address',
  city VARCHAR(30) NOT NULL COMMENT 'Name of the City',
  state VARCHAR(30) NOT NULL COMMENT 'State Name',
  zipcode VARCHAR(12) NOT NULL COMMENT 'Zipcode or Postal code',
  country_id TINYINT NOT NULL
);
```

	Time	Action	Response	Duration / Fetch Time
5	16:32:55	CREATE TABLE gb_cust (cust_id INT NOT NULL COM...	0 row(s) affected	0.026 sec

```
ALTER TABLE gb_cust ADD CONSTRAINT gb_cust_pk PRIMARY KEY ( cust_id );
```

	Time	Action	Response	Duration / Fetch Time
6	16:33:40	ALTER TABLE gb_cust ADD CONSTRAINT gb_cust_pk PRIMA...	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.042 sec

```
CREATE TABLE gb_finish (
```

```

finish_id SMALLINT NOT NULL COMMENT 'Unique finish ID',
shade    VARCHAR(30) NOT NULL COMMENT 'Name of the finish shade'
);

```

	Time	Action	Response	Duration / Fetch Time
7	16:34:19	CREATE TABLE gb_finish (finish_id SMALLINT NOT NULL...	0 row(s) affected	0.014 sec

```

ALTER TABLE gb_finish ADD CONSTRAINT sk_finish_pk PRIMARY KEY ( finish_id );

```

	Time	Action	Response	Duration / Fetch Time
8	16:34:38	ALTER TABLE gb_finish ADD CONSTRAINT sk_finish_pk PRIM...	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.039 sec

```

CREATE TABLE gb_ord_prod (
    order_id    BIGINT NOT NULL,
    prod_id     INT NOT NULL,
    finish_id   SMALLINT NOT NULL,
    quant       SMALLINT NOT NULL COMMENT 'Order quantity',
    discount_pct DECIMAL(4, 2) COMMENT 'Discount applied in percentage'
);

```

	Time	Action	Response	Duration / Fetch Time
9	16:35:01	CREATE TABLE gb_ord_prod (order_id BIGINT NOT NUL...	0 row(s) affected	0.024 sec

```

ALTER TABLE gb_ord_prod
    ADD CONSTRAINT gb_ord_prod_pk PRIMARY KEY ( order_id,
                                                prod_id,
                                                finish_id );

```

	Time	Action	Response	Duration / Fetch Time
10	16:35:35	ALTER TABLE gb_ord_prod . ADD CONSTRAINT gb_ord_pro...	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.047 sec

```

CREATE TABLE gb_order (
    order_id    BIGINT NOT NULL COMMENT 'Unique order id',
    order_date  DATETIME NOT NULL COMMENT 'Order date',
    cust_id     INT NOT NULL
);

```

	Time	Action	Response	Duration / Fetch Time
11	16:35:49	CREATE TABLE gb_order (order_id BIGINT NOT NULL CO...	0 row(s) affected	0.017 sec

```

ALTER TABLE gb_order ADD CONSTRAINT gb_order_pk PRIMARY KEY ( order_id );

```

	Time	Action	Response	Duration / Fetch Time
12	16:36:12	ALTER TABLE gb_order ADD CONSTRAINT gb_order_pk PRIM...	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.041 sec

```

CREATE TABLE gb_prod (
    prod_id    INT NOT NULL COMMENT 'Unique product id',
    name       VARCHAR(100) NOT NULL COMMENT 'Product Name',
    unit_price DECIMAL(8, 2) NOT NULL COMMENT 'Product price in USD'
);

```

);

	Time	Action	Response	Duration / Fetch Time
13	16:36:43	CREATE TABLE gb_prod (prod_id INT NOT NULL COMM...	0 row(s) affected	0.016 sec

ALTER TABLE gb_prod ADD CONSTRAINT gb_prod_pk PRIMARY KEY (prod_id);

	Time	Action	Response	Duration / Fetch Time
14	16:37:07	ALTER TABLE gb_prod ADD CONSTRAINT gb_prod_pk PRIMA...	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.032 sec

ALTER TABLE gb_cust
ADD CONSTRAINT gb_cust_gb_country_fk FOREIGN KEY (country_id)
REFERENCES gb_country (country_id);

	Time	Action	Response	Duration / Fetch Time
15	16:37:24	ALTER TABLE gb_cust ADD CONSTRAINT gb_cust_gb_cou...	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.053 sec

ALTER TABLE gb_ord_prod
ADD CONSTRAINT gb_ord_prod_gb_finish_fk FOREIGN KEY (finish_id)
REFERENCES gb_finish (finish_id);

	Time	Action	Response	Duration / Fetch Time
16	16:37:42	ALTER TABLE gb_ord_prod ADD CONSTRAINT gb_ord_pro...	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.051 sec

ALTER TABLE gb_ord_prod
ADD CONSTRAINT gb_ord_prod_gb_order_fk FOREIGN KEY (order_id)
REFERENCES gb_order (order_id);

	Time	Action	Response	Duration / Fetch Time
17	16:37:57	ALTER TABLE gb_ord_prod ADD CONSTRAINT gb_ord_pro...	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.048 sec

ALTER TABLE gb_ord_prod
ADD CONSTRAINT gb_ord_prod_gb_prod_fk FOREIGN KEY (prod_id)
REFERENCES gb_prod (prod_id);

	Time	Action	Response	Duration / Fetch Time
18	16:38:28	ALTER TABLE gb_ord_prod ADD CONSTRAINT gb_ord_pro...	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.043 sec

ALTER TABLE gb_order
ADD CONSTRAINT gb_order_gb_cust_fk FOREIGN KEY (cust_id)
REFERENCES gb_cust (cust_id);

	Time	Action	Response	Duration / Fetch Time
19	16:38:44	ALTER TABLE gb_order ADD CONSTRAINT gb_order_gb_c...	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.060 sec



1.3.2 Insert data into all business tables from the stage tables

Disable all foreign keys before we populate data into tables.

set foreign_key_checks= 0;

	Time	Action	Response	Duration / Fetch Time
68	16:52:41	set foreign_key_checks= 0	0 row(s) affected	0.00054 sec

select * from gb_country;

	Time	Action	Response	Duration / Fetch Time
69	16:53:39	select * from gb_country LIMIT 0, 50000	0 row(s) returned	0.0054 sec / 0.00001...

cust_id

frame

lname

st_address

city

state

zipcode

country_id

1000

1000

1000

1000

1000

1000

1000

1000

gb_cust 2

Apply

Reset

Action Output

	Time	Action	Response	Duration / Fetch Time
70	16:54:59	select * from gb_cust LIMIT 0, 50000	0 row(s) returned	0.0029 sec / 0.00001...

Result Grid

Filter Rows:

Search

Edit:

Export/Import:

finish_id shade

1000

1000

gb_finish 3

Apply

Reset

Action Output

	Time	Action	Response	Duration / Fetch Time
71	16:55:33	select * from gb_finish LIMIT 0, 50000	0 row(s) returned	0.0020 sec / 0.00001...

order_id

prod_id

finish_id

quant

discount_pct

PROD

PROD

PROD

PROD

PROD

gb_ord_prod 4

Apply

Reset

Action Output

0

	Time	Action	Response	Duration / Fetch Time
72	16:56:02	select * from gb_ord_prod LIMIT 0, 50000	0 row(s) returned	0.0017 sec / 0.00001...

order_id

order_date

cust_id

10000

10000

10000

gb_order 5

Apply

Reset

Action Output

1

	Time	Action	Response	Duration / Fetch Time
73	16:56:18	select * from gb_order LIMIT 0, 50000	0 row(s) returned	0.00059 sec / 0.000...

select * from gb_prod;



The screenshot shows a database management tool interface. At the top, there's a table header with columns 'prod_id', 'name', and 'unit_price'. Below it, a table named 'gb_prod 6' is displayed. The 'Action Output' section at the bottom shows a log of actions. Action 74, at 16:56:41, executed 'select * from gb_prod LIMIT 0, 50000' and returned 0 row(s) in 0.0014 seconds.

Time	Action	Response	Duration / Fetch Time
74 16:56:41	select * from gb_prod LIMIT 0, 50000	0 row(s) returned	0.0014 sec / 0.00001...

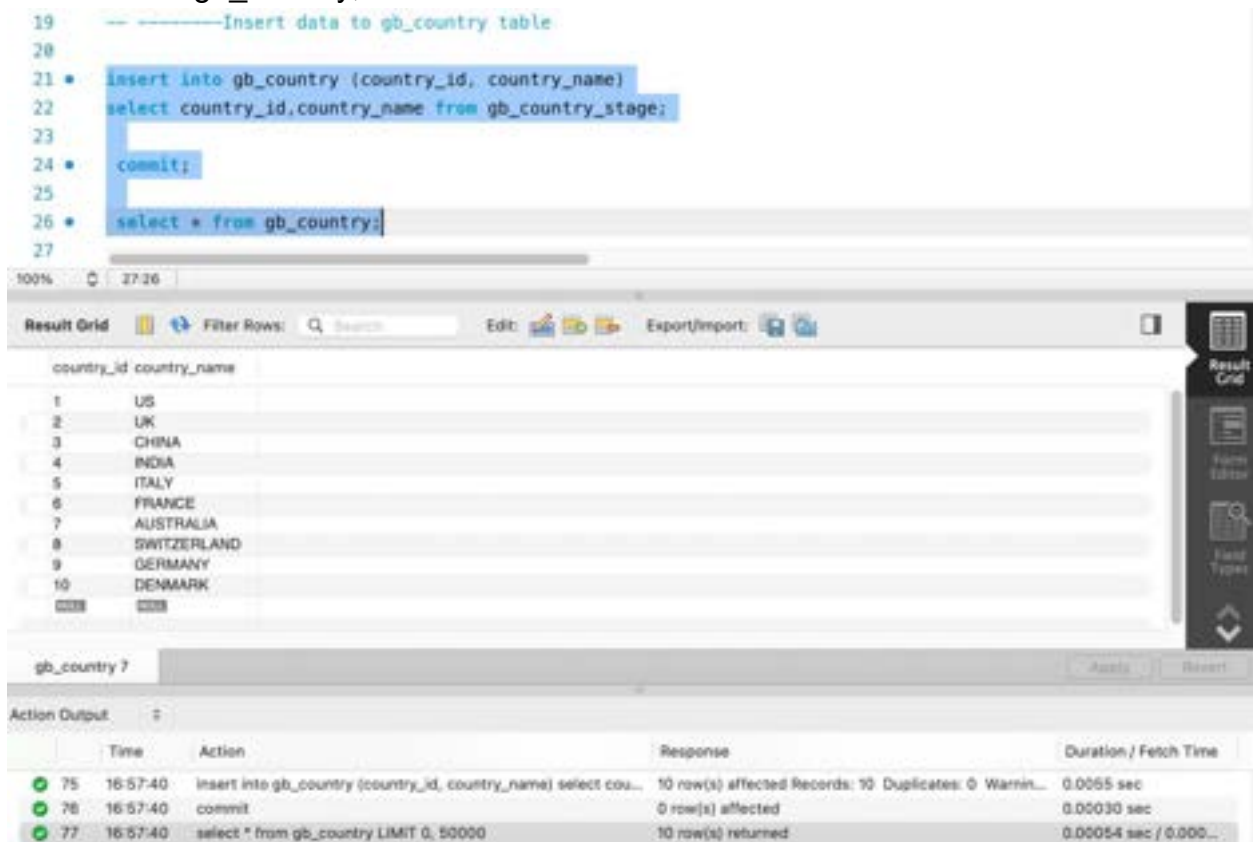
Insert data to gb_country table

insert into gb_country (country_id, country_name)

select country_id, country_name from gb_country_stage;

commit;

select * from gb_country;



The screenshot shows a database management tool interface. The top section displays a SQL script with the following lines: 19 -- Insert data to gb_country table, 20, 21 insert into gb_country (country_id, country_name), 22 select country_id, country_name from gb_country_stage;, 23, 24 commit;, 25, 26 select * from gb_country;, 27. Below the script, the 'Result Grid' shows the data for the 'gb_country' table. The table has two columns: 'country_id' and 'country_name'. The data rows are: 1 US, 2 UK, 3 CHINA, 4 INDIA, 5 ITALY, 6 FRANCE, 7 AUSTRALIA, 8 SWITZERLAND, 9 GERMANY, 10 DENMARK. The 'Action Output' section at the bottom shows a log of actions. Action 75, at 16:57:40, executed 'insert into gb_country (country_id, country_name) select cou...' and affected 10 rows in 0.0055 seconds. Action 76, at 16:57:40, executed 'commit' and affected 0 rows in 0.00030 seconds. Action 77, at 16:57:40, executed 'select * from gb_country LIMIT 0, 50000' and returned 10 row(s) in 0.00054 seconds.

```
19 -- Insert data to gb_country table
20
21 insert into gb_country (country_id, country_name)
22 select country_id, country_name from gb_country_stage;
23
24 commit;
25
26 select * from gb_country;
27
```

country_id	country_name
1	US
2	UK
3	CHINA
4	INDIA
5	ITALY
6	FRANCE
7	AUSTRALIA
8	SWITZERLAND
9	GERMANY
10	DENMARK

Time	Action	Response	Duration / Fetch Time
75 16:57:40	insert into gb_country (country_id, country_name) select cou...	10 row(s) affected Records: 10 Duplicates: 0 Warnin...	0.0055 sec
76 16:57:40	commit	0 row(s) affected	0.00030 sec
77 16:57:40	select * from gb_country LIMIT 0, 50000	10 row(s) returned	0.00054 sec / 0.000...

Insert data to gb_cust table

insert into gb_cust (cust_id, fname, lname, st_address, city, state, zipcode, country_id)

select cust_id, fname, lname, st_address, city, state, zipcode, country_id

from gb_cust_stage;

commit;

select * from gb_cust;

28 -- Insert data to gb_cust table

29

30 * insert into gb_cust (cust_id, fname, lname, st_address, city, state, zipcode, country_id)

31 select cust_id, fname, lname, st_address, city, state, zipcode, country_id

32 from gb_cust_stage;

33

34 * commit;

35

36 * select * from gb_cust;

37

100% 24:38

Result Grid

cust_id	fname	lname	st_address	city	state	zipcode	country_id
100001	MIKE	Ashton	4024 Pointe Lane	ORLANDO	FLORIDA	30741	1
100002	LUCY	Baron	2181 Big Elm	ALBANY	KENTUCKY	42301	1
100003	MAYA	Barrington	4081 Thunder Road	Addison	MAINE	02368	1
100004	KELLY	Carson	3235 Elsie Drive	CHICO	CALIFORNIA	90260	1
100005	JAMIE	Finley	3862 Hampton Meadows	Albuquerque	NEW MEXICO	55887	1
100006	NICK	Finley	4715 Cherry Camp Road	IRVING	TEXAS	77016	1
100007	HARRY	Hanson	2602 Sycamore Lake Road	BATH	VIRGINIA	23666	1
100008	Denick	Snyders	3366 Juniper Drive	EDISON	NEW JERSEY	07960	1
100009	Lon	Bonher	4852 Rosemont Avenue	ITHACA	NEW YORK	11757	1
100058	SIMRAN	KAUR	12, Mill Road	AMRITSAR	PUNJAB	143001	4
100059	VISHAL	RAVAT	301, Maharaja Pratap Co...	PUNE	MAHARASHTRA	413203	4
100060	DISHA	KHURANA	16 Prakash Apartment, An...	Sultanpur	DELHI	100041	4
100061	Chand...	Chaudhri	205, Vardhman Chamber...	Lucknow	UTTAR PRADESH	202125	4
100062	AMRIT	RAO	117, 1st Floor, Raheja Arc...	MYSORE	KARNATAKA	560002	4
100063	TARAL	JANA	47, 47, Suvaa Colony, Ba...	AHMEDABAD	GUJARAT	380001	4
100064	KAJAL	SINGH	E 18, Part2, Main Market...	AMBALA	HARYANA	121002	4
100065	HARSH	KHANNA	C/302, Baba Sadan, Rata...	PATIALA	PUNJAB	143002	4
100066	Gavino	Bove	Via Alessandro Manzoni 1...	ROME	LAZIO	27026	5
100067	Angela	Jephson	Via Croce Rossa 142	PALERMO	SICILY	8020	5
100068	Gilbert	Wolf	Lungodora Napoli 17	AISNE	PICARDIE	36022	6
100069	AYLA	ADAMS	21 Ocean Acres Drive	SYDNEY	NEW SOUTH WA...	2004	7
100070	TALIA	ALLEN	63 Ocean Street	NEWCASTLE	NEW SOUTH WA...	2001	7
100071	Charles	Patel	47973 Renee Ridges	South Whitney	Oklahoma	99731	8
100072	Daniel	Franklin	147 Foster Row	Navamshire	Ohio	92909	8
100073	Jonat...	Greer	75012 Mitchell Harbors S...	Schmidtstown	Iowa	71285	8
100074	Cristina	Garcia	964 Alex Harbors	East Shawns...	Minnesota	48624	8
100075	Jason	Henderson	1064 Lawson Parkways A...	Port Christop...	Connecticut	95522	9
100076	Jasmine	Gentry	8087 Edgar Dale	New Timmouth	Nebraska	43346	9
100077	Caitlin	Torres	33193 Sierra Glens Apt. 288	Kimberlyville	New Mexico	58954	9
100078	Michael	Conner	88394 Taylor Rue	Heatherdown	Vermont	41557	9
100079	Joshua	Bell	83666 Price Valley Suite...	Lake Tracy	New Mexico	88068	9
100080	Raym...	Meyer	0620 Mopoe Shoals Apt...	West Mary	Maine	66090	10
100081	Juan	Parrish	83221 Knight Pass Suite...	New Tarabor...	South Carolina	43369	10
100082	Jeffery	Coleman	746 Tammy Center	Charlestown	Michigan	57595	10
100083	Amy	King	37437 David Forks	Curtismouth	North Dakota	19620	10

gb_cust 8

Action Output

	Time	Action	Response	Duration / Fetch Time
80	17:00:59	insert into gb_cust (cust_id, fname, lname, st_address, city, s...	83 row(s) affected Records: 83 Duplicates: 0 Warnin...	0.0076 sec
81	17:00:59	commit	0 row(s) affected	0.00084 sec
82	17:00:59	select * from gb_cust LIMIT 0, 50000	83 row(s) returned	0.00068 sec / 0.500...

Insert data to gb_finish table

```

insert into gb_finish (finish_id, shade)
select finish_id, shade
from gb_finish_stage;
commit;
select * from gb_finish;

```

38 ————Insert data to gb_finish table
 39
 40 * insert into gb_finish (finish_id, shade)
 41 select finish_id, shade
 42 from gb_finish_stage;
 43
 44 * commit;
 45
 46 * select * from gb_finish;
 47

100% 26:48

Result Grid

finish_id	shade
100	Cherry
101	Classic Gray
102	Driftwood
103	Fruitwood
104	Golden Oak
105	Green Velvet
106	Gunstock
107	Natural
108	Natural Ash
109	Oak
110	Provincial
111	Puritan Pine
112	Red Chestnut
113	Simply White
114	Honey Maple
115	Dark Walnut
116	Black Cherry
117	Barn Red
118	Vintage Aqua
119	Spring Oak

gb_finish 9

Action Output

	Time	Action	Response	Duration / Fetch Time
83	17:02:41	insert into gb_finish (finish_id, shade) select finish_id, shade...	20 row(s) affected Records: 20 Duplicates: 0 Warnin...	0.0026 sec
84	17:02:41	commit	0 row(s) affected	0.00036 sec
85	17:02:41	select * from gb_finish LIMIT 0, 50000	20 row(s) returned	0.00045 sec / 0.000...

Insert data to gb_ord_prod table

insert into gb_ord_prod (order_id, prod_id, finish_id, quant, discount_pct)

select order_id, prod_id, finish_id, quant, discount_pct

from gb_ord_prod_stage;

commit;

select * from gb_ord_prod;

The screenshot displays a database management interface. At the top, a SQL script is shown with the following commands:

```
48 -- -----Insert data to gb_ord_prod table
49
50 * insert into gb_ord_prod (order_id, prod_id, finish_id, quant, discount_pct)
51   select order_id, prod_id, finish_id, quant, discount_pct
52   from gb_ord_prod_stage;
53
54 * commit;
55
56 * select * from gb_ord_prod;
57
```

Below the script, the 'Result Grid' shows the output of the SQL execution. The grid has five columns: order_id, prod_id, finish_id, quant, and discount_pct. It contains 300 rows of data, with the first few rows visible as follows:

order_id	prod_id	finish_id	quant	discount_pct
100000001	10001	100	2	10.00
100000001	10025	101	4	13.00
100000001	10025	119	5	35.00
100000002	10001	100	2	5.00
100000002	10002	101	1	5.05
100000002	10009	102	1	18.00
100000003	10002	101	1	10.00
100000003	10003	102	1	20.00
100000003	10010	110	2	19.00
100000142	10018	104	4	50.00
100000143	10007	117	3	5.00
100000143	10018	105	3	35.00
100000144	10007	118	1	10.00
100000144	10018	106	1	20.00
100000145	10007	119	1	15.00
100000145	10018	107	1	15.00
100000146	10007	100	2	19.00
100000146	10018	108	4	20.00
100000147	10007	101	3	45.00
100000147	10018	109	3	45.00
100000148	10007	102	6	20.00
100000148	10018	110	6	15.00
100000149	10007	103	5	60.00
100000149	10018	100	5	10.00
100000150	10002	101	4	15.00
100000150	10002	104	4	20.00
100000150	10007	104	6	65.00
100000150	10011	115	2	45.00
100000150	10018	101	3	50.00
100000151	10002	102	3	10.00
100000151	10007	105	1	20.00
100000151	10018	102	1	15.00
100000152	10002	103	1	50.00
100000152	10007	106	1	21.00
100000152	10018	103	5	20.00

At the bottom, the 'Action Output' section shows the execution details for the three SQL statements:

	Time	Action	Response	Duration / Fetch Time
86	17:05:08	insert into gb_ord_prod (order_id, prod_id, finish_id, quant, d...	300 row(s) affected Records: 300 Duplicates: 0 War...	0.013 sec
87	17:05:08	commit	0 row(s) affected	0.00027 sec
88	17:05:08	select * from gb_ord_prod LIMIT 0, 50000	300 row(s) returned	0.0018 sec / 0.00005...

Insert data to gb_order table

```

insert into gb_order (order_id, order_date, cust_id)
select order_id, str_to_date(order_date,'%m/%d/%Y'), cust_id
from gb_order_stage;
commit;
select * from gb_order;

```

58 -----Insert data to gb_order table
59
60 * insert into gb_order (order_id, order_date, cust_id)
61 select order_id, str_to_date(order_date,'%m/%d/%Y'), cust_id
62 from gb_order_stage;
63
64 * commit;
65
66 * select * from gb_order;
67

100% 25.68

Result Grid

order_id	order_date	cust_id
100000001	2021-06-11 00:00:00	100001
100000002	2019-07-12 00:00:00	100002
100000003	2022-04-13 00:00:00	100003
100000004	2022-03-14 00:00:00	100004
100000005	2021-10-15 00:00:00	100005
100000006	2019-11-19 00:00:00	100006
100000007	2021-03-17 00:00:00	100007
100000008	2022-07-18 00:00:00	100008
100000009	2022-10-10 00:00:00	100009
100000128	2019-02-05 00:00:00	100072
100000129	2019-01-12 00:00:00	100073
100000130	2019-05-17 00:00:00	100074
100000131	2019-01-11 00:00:00	100075
100000132	2021-10-15 00:00:00	100076
100000133	2021-04-17 00:00:00	100077
100000134	2019-11-19 00:00:00	100078
100000135	2021-03-17 00:00:00	100079
100000136	2022-10-10 00:00:00	100080
100000137	2018-01-05 00:00:00	100081
100000138	2021-02-07 00:00:00	100082
100000139	2022-02-09 00:00:00	100083
100000140	2019-02-05 00:00:00	100071
100000141	2019-01-12 00:00:00	100072
100000142	2019-05-17 00:00:00	100073
100000143	2019-01-11 00:00:00	100074
100000144	2021-10-15 00:00:00	100075
100000145	2021-04-17 00:00:00	100076
100000146	2019-11-19 00:00:00	100077
100000147	2021-03-17 00:00:00	100078
100000148	2022-10-10 00:00:00	100079
100000149	2022-07-13 00:00:00	100080
100000150	2022-04-16 00:00:00	100081
100000151	2019-06-18 00:00:00	100082
100000152	2020-10-23 00:00:00	100083
100000153	2024-04-16 00:00:00	100081

gb_order 11

Action Output

	Time	Action	Response	Duration / Fetch Time
89	17:07:00	insert into gb_order (order_id, order_date, cust_id) select or...	153 row(s) affected Records: 153 Duplicates: 0 Warn...	0.0099 sec
90	17:07:00	commit	0 row(s) affected	0.00021 sec
91	17:07:00	select * from gb_order LIMIT 0, 50000	153 row(s) returned	0.00085 sec / 0.000...

Insert data to gb_prod table

```

insert into gb_prod (prod_id, name, unit_price)
select prod_id, name, unit_price
from gb_prod_stage;
commit;
select * from gb_prod;

```

68 -----Insert data to gb_prod table

69

70 • insert into gb_prod (prod_id, name, unit_price)

71 select prod_id, name, unit_price

72 from gb_prod_stage;

73

74 • commit;

75

76 • select * from gb_prod;

77

100% 24/76

Result Grid Filter Rows: Search Edit Export/Import

prod_id	name	unit_price
10001	Cradle Bed	200.00
10002	Cabinets	345.00
10003	Couch	3000.00
10004	Entertainment Center	4500.00
10005	Writer Desk	350.00
10006	Settee	400.00
10007	Dinning Table	1100.00
10008	Coffee Table	370.00
10009	Stool	100.00
10010	Height Bar Stool	250.00
10011	Dinning Chair	150.00
10012	Accent Chair	425.00
10013	Cafe Chair	175.00
10014	Futon Chair	205.00
10015	Wardrobe	2500.00
10016	Washstand	355.00
10017	Bookcase	650.00
10018	Hair Dresser	845.00
10019	Rolltop Desk	480.00
10020	Armoire	2100.00
10021	Winebar Chair	340.00
10022	Nightstand	230.00
10023	TV Stand	550.00
10024	Futon Bed	1800.00
10025	School Chair	75.00
10026		
10027		
10028		
10029		
10030		
10031		
10032		
10033		
10034		
10035		
10036		
10037		
10038		
10039		
10040		
10041		
10042		
10043		
10044		
10045		
10046		
10047		
10048		
10049		
10050		
10051		
10052		
10053		
10054		
10055		
10056		
10057		
10058		
10059		
10060		
10061		
10062		
10063		
10064		
10065		
10066		
10067		
10068		
10069		
10070		
10071		
10072		
10073		
10074		
10075		
10076		
10077		
10078		
10079		
10080		
10081		
10082		
10083		
10084		
10085		
10086		
10087		
10088		
10089		
10090		
10091		
10092		
10093		
10094		
10095		
10096		
10097		
10098		
10099		
10100		
10101		
10102		
10103		
10104		
10105		
10106		
10107		
10108		
10109		
10110		
10111		
10112		
10113		
10114		
10115		
10116		
10117		
10118		
10119		
10120		
10121		
10122		
10123		
10124		
10125		
10126		
10127		
10128		
10129		
10130		
10131		
10132		
10133		
10134		
10135		
10136		
10137		
10138		
10139		
10140		
10141		
10142		
10143		
10144		
10145		
10146		
10147		
10148		
10149		
10150		
10151		
10152		
10153		
10154		
10155		
10156		
10157		
10158		
10159		
10160		
10161		
10162		
10163		
10164		
10165		
10166		
10167		
10168		
10169		
10170		
10171		
10172		
10173		
10174		
10175		
10176		
10177		
10178		
10179		
10180		
10181		
10182		
10183		
10184		
10185		
10186		
10187		
10188		
10189		
10190		
10191		
10192		
10193		
10194		
10195		
10196		
10197		
10198		
10199		
10200		
10201		
10202		
10203		
10204		
10205		
10206		
10207		
10208		
10209		
10210		
10211		
10212		
10213		
10214		
10215		
10216		
10217		
10218		
10219		
10220		
10221		
10222		
10223		
10224		
10225		
10226		
10227		
10228		
10229		
10230		
10231		
10232		
10233		
10234		
10235		
10236		
10237		
10238		
10239		
10240		
10241		
10242		
10243		
10244		
10245		
10246		
10247		
10248		
10249		
10250		
10251		
10252		
10253		
10254		
10255		
10256		
10257		
10258		
10259		
10260		
10261		
10262		
10263		
10264		
10265		
10266		
10267		
10268		
10269		
10270		
10271		
10272		
10273		
10274		
10275		
10276		
10277		
10278		
10279		
10280		
10281		
10282		
10283		
10284		
10285		
10286		
10287		
10288		
10289		
10290		
10291		
10292		
10293		
10294		
10295		
10296		
10297		
10298		
10299		
10300		
10301		
10302		
10303		
10304		
10305		
10306		
10307		
10308		
10309		
10310		
10311		
10312		
10313		
10314		
10315		
10316		
10317		
10318		
10319		
10320		
10321		
10322		
10323		
10324		
10325		
10326		
10327		
10328		
10329		
10330		
10331		
10332		
10333		
10334		
10335		
10336		
10337		
10338		
10339		
10340		
10341		
10342		
10343		
10344		
10345		
10346		
10347		
10348		
10349		
10350		
10351		
10352		
10353		
10354		
10355		
10356		
10357		
10358		
10359		
10360		
10361		
10362		
10363		
10364		
10365		
10366		
10367		
10368		
10369		
10370		
10371		
10372		
10373		
10374		
10375		
10376		
10377		
10378		
10379		
10380		
10381		
10382		
10383		
10384		
10385		
10386		
10387		
10388		
10389		
10390		
10391		
10392		
10393		
10394		
10395		
10396		
10397		
10398		
10399		
10400		
10401		
10402		
10403		
10404		
10405		
10406		
10407		
10408		
10409		
10410		
10411		
10412		
10413		
10414		
10415		
10416		
10417		
10418		
10419		
10420		
10421		
10422		
10423		
10424		
10425		
10426		
10427		
10428		
10429		
10430		
10431		
10432		
10433		
10434		
10435		
10436		
10437		
10438		
10439		
10440		
10441		
10442		
10443		
10444		
10445		
10446		
10447		
10448		
10449		
10450		
10451		
10452		
10453		
10454		
10455		
10456		
10457		
10458		
10459		
10460		
10461		
10462		
10463		
10464		
10465		
10466		
10467		
10468		
10469		
10470		
10471		
10472		
10473		
10474		
10475		
10476		
10477		
10478		
10479		
10480		
10481		
10482		
10483		
10484		
10485		
10486		
10487		
10488		
10489		
10490		
10491		
10492		
10493		
10494		
10495		
10496		
10497		
10498		
10499		
10500		
10501		
10502		
10503		
10504		
10505		
10506		
10507		
10508		
10509		
10510		
10511		
10512		
10513		
10514		

1.4 Screenshots of queries and corresponding results of all data dictionary queries, and record count queries and their result

1.4.1 REM, List of Tables

```
3  -- REM, List of Tables
4
5  * select table_name, table_schema
6    from information_schema.tables
7   where table_name like 'gb_%' and table_schema='gb_pvf';
8
```

The screenshot shows the results of a SQL query in a web application. The query is: `select table_name, table_schema from information_schema.tables where table_name like 'gb_%' and table_schema='gb_pvf';`. The results are displayed in a table with two columns: `TABLE_NAME` and `TABLE_SCHEMA`. The table contains 12 rows of data, all with `gb_pvf` as the schema. The application interface includes a 'Result Grid' header, a search bar, and an 'Export' button. On the right side, there are icons for 'Result Grid', 'Form Editor', 'Field Types', and 'Query Stats'. At the bottom, there is an 'Action Output' section showing the execution details: '1 17:16:51 select table_name, table_schema from information_schema.L... 12 row(s) returned 0.0014 sec / 0.00001...'.

TABLE_NAME	TABLE_SCHEMA
gb_country	gb_pvf
gb_country_stage	gb_pvf
gb_cust	gb_pvf
gb_cust_stage	gb_pvf
gb_finish	gb_pvf
gb_finish_stage	gb_pvf
gb_ord_prod	gb_pvf
gb_ord_prod_stage	gb_pvf
gb_order	gb_pvf
gb_order_stage	gb_pvf
gb_prod	gb_pvf
gb_prod_stage	gb_pvf

1.4.2 REM, List of Table Columns

```
9  -- REM, List of Table Columns
10
11  * select tab.table_schema as database_schema,
12         tab.table_name as table_name,
13         col.ordinal_position as column_id,
14         col.column_name as column_name,
15         col.data_type as data_type,
16         col.CHARACTER_MAXIMUM_LENGTH,
17         col.COLUMN_TYPE,
18         col.column_key
19   from information_schema.tables as tab inner join information_schema.columns as col on tab.table_name=col.tab
20  where tab.table_name like 'gb_%' and tab.table_schema='gb_pvf'
21  order by table_name;
22
```

The screenshot shows the results of a SQL query in a web application. The query is: `select tab.table_schema as database_schema, tab.table_name as table_name, col.ordinal_position as column_id, col.column_name as column_name, col.data_type as data_type, col.CHARACTER_MAXIMUM_LENGTH, col.COLUMN_TYPE, col.column_key from information_schema.tables as tab inner join information_schema.columns as col on tab.table_name=col.tab where tab.table_name like 'gb_%' and tab.table_schema='gb_pvf' order by table_name;`. The results are displayed in a table with 8 columns: `database_sche...`, `table_name`, `column_id`, `column_name`, `data_type`, `CHARACTER_MAXIMUM_LENGTH`, `COLUMN_TYPE`, and `COLUMN_KEY`. The table contains 9 rows of data. The application interface includes a 'Result Grid' header, a search bar, and an 'Export' button. On the right side, there are icons for 'Result Grid', 'Form Editor', and 'Field Types'. At the bottom, there is an 'Action Output' section showing the execution details: '1 17:16:51 select table_name, table_schema from information_schema.L... 12 row(s) returned 0.0014 sec / 0.00001...'.

database_sche...	table_name	column_id	column_name	data_type	CHARACTER_MAXIMUM_LENGTH	COLUMN_TYPE	COLUMN_KEY
gb_pvf	gb_country	1	country_id	tinyint	4	tinyint	PRI
gb_pvf	gb_country	2	country_name	varchar	50	varchar(50)	
gb_pvf	gb_country_stage	1	country_id	text	65535	text	
gb_pvf	gb_country_stage	2	country_name	text	65535	text	
gb_pvf	gb_cust	1	cust_id	int	4	int	PRI
gb_pvf	gb_cust	2	frame	varchar	30	varchar(30)	
gb_pvf	gb_cust	3	lname	varchar	30	varchar(30)	

gb_pvfc	gb_cust	4	st_address	varchar	100	varchar(100)		
gb_pvfc	gb_cust	5	city	varchar	30	varchar(30)		
gb_pvfc	gb_cust	6	state	varchar	30	varchar(30)		
gb_pvfc	gb_cust	7	zipcode	varchar	12	varchar(12)		
gb_pvfc	gb_cust	8	country_id	tinyint	0000	tinyint	MUL	
gb_pvfc	gb_cust_stage	1	cust_id	text	65535	text		
gb_pvfc	gb_cust_stage	2	fname	text	65535	text		
gb_pvfc	gb_cust_stage	3	lname	text	65535	text		
gb_pvfc	gb_cust_stage	4	st_address	text	65535	text		
gb_pvfc	gb_cust_stage	5	city	text	65535	text		
gb_pvfc	gb_cust_stage	6	state	text	65535	text		
gb_pvfc	gb_cust_stage	7	zipcode	text	65535	text		
gb_pvfc	gb_cust_stage	8	country_id	text	65535	text		
gb_pvfc	gb_finish	1	finish_id	smallint	0000	smallint	PR	
gb_pvfc	gb_finish	2	shade	varchar	30	varchar(30)		
gb_pvfc	gb_finish_stage	2	shade	text	65535	text		
gb_pvfc	gb_finish_stage	1	finish_id	text	65535	text		
gb_pvfc	gb_ord_prod	1	order_id	bigint	0000	bigint	PR	
gb_pvfc	gb_ord_prod	2	prod_id	int	0000	int	PR	
gb_pvfc	gb_ord_prod	3	finish_id	smallint	0000	smallint	PR	
gb_pvfc	gb_ord_prod	4	quant	smallint	0000	smallint		
gb_pvfc	gb_ord_prod	5	discount_pct	decimal	0000	decimal(4,2)		
gb_pvfc	gb_ord_prod_st...	1	order_id	text	65535	text		
gb_pvfc	gb_ord_prod_st...	2	prod_id	text	65535	text		
gb_pvfc	gb_ord_prod_st...	3	finish_id	text	65535	text		
gb_pvfc	gb_ord_prod_st...	4	discount_pct	text	65535	text		
gb_pvfc	gb_ord_prod_st...	5	quant	text	65535	text		
gb_pvfc	gb_order	1	order_id	bigint	0000	bigint	PR	
gb_pvfc	gb_order	2	order_date	datetime	0000	datetime		
gb_pvfc	gb_order	3	cust_id	int	0000	int	MUL	
gb_pvfc	gb_order_stage	1	order_id	text	65535	text		
gb_pvfc	gb_order_stage	2	order_date	text	65535	text		
gb_pvfc	gb_order_stage	3	cust_id	text	65535	text		
gb_pvfc	gb_prod	1	prod_id	int	0000	int	PR	
gb_pvfc	gb_prod	2	name	varchar	100	varchar(100)		
gb_pvfc	gb_prod	3	unit_price	decimal	0000	decimal(8,2)		
gb_pvfc	gb_prod_stage	1	prod_id	text	65535	text		
gb_pvfc	gb_prod_stage	2	name	text	65535	text		
gb_pvfc	gb_prod_stage	3	unit_price	text	65535	text		

Result 3

Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
2	17:20:05	select tab.table_schema as database_schema, tab.table_na...	46 row(s) returned	0.037 sec / 0.000030...

1.4.3 REM, List of Table Column Constraints

```
23 -- REM, List of Table Column Constraints
24
25 * select *
26 from information_schema.table_constraints
27 where table_name like 'gb_5' and table_schema= 'gb_pvt';
28 order by table_name;
29
```

100% 21:28

Result Grid Filter Rows: Search Export:

	CONSTRAINT_CATALOG	CONSTRAINT_SCHEMA	CONSTRAINT_NAME	TABLE_SCHEMA	TABLE_NAME	CONSTRAINT_TYPE	ENFORCED
def		gb_pvt	PRIMARY	gb_pvt	gb_country	PRIMARY KEY	YES
def		gb_pvt	PRIMARY	gb_pvt	gb_cust	PRIMARY KEY	YES
def		gb_pvt	gb_cust_gb_country_fk	gb_pvt	gb_cust	FOREIGN KEY	YES
def		gb_pvt	PRIMARY	gb_pvt	gb_finish	PRIMARY KEY	YES
def		gb_pvt	PRIMARY	gb_pvt	gb_ord_prod	PRIMARY KEY	YES
def		gb_pvt	gb_ord_prod_gb_finish_fk	gb_pvt	gb_ord_prod	FOREIGN KEY	YES
def		gb_pvt	gb_ord_prod_gb_order_fk	gb_pvt	gb_ord_prod	FOREIGN KEY	YES
def		gb_pvt	gb_ord_prod_gb_prod_fk	gb_pvt	gb_ord_prod	FOREIGN KEY	YES
def		gb_pvt	PRIMARY	gb_pvt	gb_order	PRIMARY KEY	YES
def		gb_pvt	gb_order_gb_cust_fk	gb_pvt	gb_order	FOREIGN KEY	YES
def		gb_pvt	PRIMARY	gb_pvt	gb_prod	PRIMARY KEY	YES

table_constraints 4 Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
3	17:22:46	select * from information_schema.table_constraints where ta...	11 row(s) returned	0.023 sec / 0.000021...

1.4.4 REM, List of Table Column Comments

select count(*) from gb_country;

```

31  -- REM, List of Table Column Comments
32
33  * select table_schema,table_name,ordinal_position,column_name, column_comment
34    from information_schema.columns
35   where table_name like 'gb_%' and table_schema='gb_pvt%'
36   order by 1,2,3;
37

```

100% 16:36

Result Grid

TABLE_SCHEMA	TABLE_NAME	ORDINAL_POSITION	COLUMN_NAME	COLUMN_COMMENT
gb_pvtc	gb_country	1	country_id	Unique Country ID
gb_pvtc	gb_country	2	country_name	Name of the country
gb_pvtc	gb_country_stage	1	country_id	
gb_pvtc	gb_country_stage	2	country_name	
gb_pvtc	gb_cust	1	cust_id	Unique customer id
gb_pvtc	gb_cust	2	fname	First name of the customer
gb_pvtc	gb_cust	3	lname	Last name of the customer
gb_pvtc	gb_cust	4	st_address	Street address
gb_pvtc	gb_cust	5	city	Name of the City
gb_pvtc	gb_cust	6	state	State Name
gb_pvtc	gb_cust	7	zipcode	Zipcode or Postal code
gb_pvtc	gb_cust	8	country_id	
gb_pvtc	gb_cust_stage	1	cust_id	
gb_pvtc	gb_cust_stage	2	fname	
gb_pvtc	gb_cust_stage	3	lname	
gb_pvtc	gb_cust_stage	5	city	
gb_pvtc	gb_cust_stage	6	state	
gb_pvtc	gb_cust_stage	7	zipcode	
gb_pvtc	gb_cust_stage	8	country_id	
gb_pvtc	gb_finish	1	finish_id	Unique finish ID
gb_pvtc	gb_finish	2	shade	Name of the finish shade
gb_pvtc	gb_finish_stage	1	finish_id	
gb_pvtc	gb_finish_stage	2	shade	
gb_pvtc	gb_ord_prod	1	order_id	
gb_pvtc	gb_ord_prod	2	prod_id	
gb_pvtc	gb_ord_prod	3	finish_id	
gb_pvtc	gb_ord_prod	4	quant	Order quantity
gb_pvtc	gb_ord_prod	5	discount_pct	Discount applied in perce...
gb_pvtc	gb_ord_prod_st...	1	order_id	
gb_pvtc	gb_ord_prod_st...	2	prod_id	
gb_pvtc	gb_ord_prod_st...	3	finish_id	
gb_pvtc	gb_ord_prod_st...	4	discount_pct	
gb_pvtc	gb_ord_prod_st...	5	quant	
gb_pvtc	gb_order	1	order_id	Unique order id
gb_pvtc	gb_order	2	order_date	Order date
gb_pvtc	gb_order	3	cust_id	
gb_pvtc	gb_order_stage	1	order_id	
gb_pvtc	gb_order_stage	2	order_date	
gb_pvtc	gb_order_stage	3	cust_id	
gb_pvtc	gb_prod	1	prod_id	Unique product id
gb_pvtc	gb_prod	2	name	Product Name
gb_pvtc	gb_prod	3	unit_price	Product price in USD
gb_pvtc	gb_prod_stage	1	prod_id	
gb_pvtc	gb_prod_stage	2	name	
gb_pvtc	gb_prod_stage	3	unit_price	

columns 5

Action Output

Time	Action	Response	Duration / Fetch Time
17:23:42	select table_schema,table_name,ordinal_position,column_na...	46 row(s) returned	0.013 sec / 0.000021...

1.4.5 Retrieving number records for each table

```
39 -- Retrieving number records for each table (should be Zero since we haven't yet inserted data to tables)
40 * SELECT table_name, table_rows
41 FROM INFORMATION_SCHEMA.TABLES
42 WHERE TABLE_SCHEMA = 'gb_pufs';
43
```

The screenshot shows a database client interface with a query result grid. The grid has two columns: TABLE_NAME and TABLE_ROWS. The data is as follows:

TABLE_NAME	TABLE_ROWS
gb_country	10
gb_country_stage	10
gb_cust	83
gb_cust_stage	83
gb_fresh	20
gb_fresh_stage	20
gb_ord_prod	300
gb_ord_prod_stage	300
gb_order	153
gb_order_stage	153
gb_prod	25
gb_prod_stage	25

The interface also shows an Action Output section with the following entry:

Time	Action	Response	Duration / Fetch Time
17:25:15	SELECT table_name, table_rows FROM INFORMATION_SCHE...	12 row(s) returned	0.038 sec / 0.000016...

1.4.6 Record Counts of each tables.

select count(*) from gb_country;

```
46 * select count(*) from gb_country;
47
```

The screenshot shows a database client interface with a query result grid. The grid has one column: count(*). The data is as follows:

count(*)
10

The interface also shows an Action Output section with the following entry:

Time	Action	Response	Duration / Fetch Time
17:16:02	select count(*) from gb_country LIMIT 0, 50000	1 row(s) returned	0.011 sec / 0.000012...

select count(*) from gb_cust;

The screenshot shows a database query interface. At the top, a SQL query is entered: `select count(*) from gb_cust;`. Below the query, the 'Result Grid' displays a single row with the value 83. The 'Action Output' table at the bottom shows the execution details.

	Time	Action	Response	Duration / Fetch Time
128	17:17:11	select count(*) from gb_cust LIMIT 0, 50000	1 row(s) returned	0.0018 sec / 0.00001...

select count(*) from gb_finish;

The screenshot shows a database query interface. At the top, a SQL query is entered: `select count(*) from gb_finish;`. Below the query, the 'Result Grid' displays a single row with the value 20. The 'Action Output' table at the bottom shows the execution details.

	Time	Action	Response	Duration / Fetch Time
129	17:17:37	select count(*) from gb_finish LIMIT 0, 50000	1 row(s) returned	0.0054 sec / 0.00001...

select count(*) from gb_ord_prod;

The screenshot shows a database query interface. At the top, a SQL query is entered: `select count(*) from gb_ord_prod;`. Below the query, the 'Result Grid' displays a single row with the value 300. The 'Action Output' table at the bottom shows the execution details.

	Time	Action	Response	Duration / Fetch Time
130	17:18:02	select count(*) from gb_ord_prod LIMIT 0, 50000	1 row(s) returned	0.0010 sec / 0.00001...

```
select count(*) from gb_order;
```

The screenshot shows a database query interface. At the top, the SQL query `select count(*) from gb_order;` is entered. Below the query, the 'Result Grid' displays a single row with the value '153'. The 'Action Output' section at the bottom shows a log entry for action 131, executed at 17:18:25, with the response '1 row(s) returned' and a duration of '0.00080 sec / 0.000...'.

Time	Action	Response	Duration / Fetch Time
131 17:18:25	select count(*) from gb_order LIMIT 0, 50000	1 row(s) returned	0.00080 sec / 0.000...

```
select count(*) from gb_prod;
```

The screenshot shows a database query interface. At the top, the SQL query `select count(*) from gb_prod;` is entered. Below the query, the 'Result Grid' displays a single row with the value '25'. The 'Action Output' section at the bottom shows a log entry for action 132, executed at 17:18:48, with the response '1 row(s) returned' and a duration of '0.00029 sec / 0.00001...'.

Time	Action	Response	Duration / Fetch Time
132 17:18:48	select count(*) from gb_prod LIMIT 0, 50000	1 row(s) returned	0.00029 sec / 0.00001...

2. Content of Project Phase 2

2.1 Three table joins SQL queries (minimum 3 table joins)

2.1.1 Table join 1

Business question:

For each PVFC product, which countries demonstrate genuine demand (sales exceeding 10 units) sufficient to justify further market investment, thereby qualifying as key markets for marketing efforts?

SQL:

```
SELECT p.prod_id,  
       p.name AS product_name,  
       c.country_name,  
       SUM(op.quant) AS total_quantity  
FROM gb_prod p  
JOIN gb_ord_prod op ON p.prod_id = op.prod_id
```

```

JOIN gb_order o ON op.order_id = o.order_id
JOIN gb_cust cu ON o.cust_id = cu.cust_id
JOIN gb_country c ON cu.country_id = c.country_id
GROUP BY p.prod_id, p.name, c.country_name
HAVING SUM(op.quant) > 10
ORDER BY total_quantity DESC, c.country_name;

```

Result screenshot:

The screenshot shows a SQL query editor with the following query:

```

40 * SELECT p.prod_id,
41      p.name AS product_name,
42      c.country_name,
43      SUM(op.quant) AS total_quantity
44 FROM gb_prod p
45 JOIN gb_ord_prod op ON p.prod_id = op.prod_id
46 JOIN gb_order o ON op.order_id = o.order_id
47 JOIN gb_cust cu ON o.cust_id = cu.cust_id
48 JOIN gb_country c ON cu.country_id = c.country_id
49 GROUP BY p.prod_id, p.name, c.country_name
50 HAVING SUM(op.quant) > 10
51 ORDER BY total_quantity DESC, c.country_name;
52

```

The results are displayed in a grid with the following columns: prod_id, product_name, country_name, total_quantity. The results are sorted by total_quantity in descending order, and then by country_name.

prod_id	product_name	country_name	total_quantity
10011	Dinning Chair	US	25
10007	Dinning Table	GERMANY	23
10014	Futon Chair	GERMANY	23
10005	Witter Desk	GERMANY	22
10007	Dinning Table	SWITZERLAND	22
10006	Settee	DENMARK	19
10018	Hair Dresser	SWITZERLAND	19
10007	Dinning Table	DENMARK	18
10006	Settee	SWITZERLAND	17
10009	Stool	US	16
10021	Windsor Chair	DENMARK	15
10016	Washstand	GERMANY	15
10018	Hair Dresser	GERMANY	15
10025	School Chair	US	15
10010	Height Bar Stool	DENMARK	14
10018	Hair Dresser	DENMARK	14
10005	Witter Desk	SWITZERLAND	14
10024	Futon Bed	SWITZERLAND	14
10023	TV Stand	US	13
10002	Cabinets	DENMARK	12
10010	Height Bar Stool	GERMANY	12
10021	Windsor Chair	GERMANY	12
10005	Witter Desk	US	12
10004	Entertainment...	US	12
10004	Entertainment...	GERMANY	11
10014	Futon Chair	SWITZERLAND	11
10017	Bookcase	US	11
10012	Accent Chair	US	11
10010	Height Bar Stool	US	11
10003	Couch	US	11
10022	Nightstand	US	11
10002	Cabinets	US	11
10001	Cradle Bed	US	11

The screenshot also shows the 'Result Grid' tab selected, and the 'Action Output' tab showing the execution details:

Time	Action	Response	Duration / Fetch Time
03:08:14	SELECT p.prod_id, p.name AS product_name, c.country_name,...	33 row(s) returned	0.0061 sec / 0.00001...

2.1.2 Table join 2

Business question:

Analyze detailed sales by geography and product attributes.

SQL:

WHW:

- What: Show order lines with country, customer, product, finish, quantity, discount and line amount.
- How: Join country, customer, order, order line, product, and finish tables.

SELECT

```
co.country_name,  
c.cust_id,  
c.fname,  
c.lname,  
o.order_id,  
o.order_date,  
p.prod_id,  
p.name      AS prod_name,  
f.finish_id,  
f.shade     AS finish_shade,  
op.quant,  
p.unit_price,  
op.discount_pct,  
(op.quant * p.unit_price * (1 - op.discount_pct/100)) AS line_amount
```

FROM gb_country co

JOIN gb_cust c ON co.country_id = c.country_id

JOIN gb_order o ON c.cust_id = o.cust_id

JOIN gb_ord_prod op ON o.order_id = op.order_id

JOIN gb_prod p ON op.prod_id = p.prod_id

JOIN gb_finish f ON op.finish_id = f.finish_id

ORDER BY

```
co.country_name,  
c.cust_id,  
o.order_id,  
p.prod_id;
```

Result screenshot:

```

29 --
30 --> SQL: 3
31 --
32 -- What: Show order lines with country, customer, product, finish, quantity, discount and line amount.
33 -- How: Join country, customer, order, order line, product, and finish tables.
34 -- Why: To surface detailed sales by geography and product attributes.
35 --
36 -- SELECT
37 --     co.country_name,
38 --     c.cust_id,
39 --     c.fname,
40 --     c.lname,
41 --     o.order_id,
42 --     o.order_date,
43 --     p.prod_id,
44 --     p.name AS prod_name,
45 --     f.finish_id,
46 --     f.shade AS finish_shade,
47 --     op.quantity,
48 --     p.unit_price,
49 --     op.discount_pct,
50 --     (op.quantity * p.unit_price * (1 - op.discount_pct/100)) AS line_amount
51 -- FROM   gb_country co
52 -- 2025 gb_cust c ON co.country_id = c.country_id
53 -- 2025 gb_order o ON c.cust_id = o.cust_id
54 -- 2025 gb_ord_prod op ON o.order_id = op.order_id
55 -- 2025 gb_prod p ON op.prod_id = p.prod_id
56 -- 2025 gb_finish f ON op.finish_id = f.finish_id
57 -- GROUP BY
58 --     co.country_name,
59 --     c.cust_id,
60 --     o.order_id,
61 --     p.prod_id
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

SQL Output: 0

Time	Action	Response	Duration / Fetch Time
19:58:25	SQL: 3	300 rows returned	0.005 sec / 0.0007s...

country_name	cust_id	fname	lname	order_id	order_date	prod_id	prod_name	finish_id	finish_shade	quant	unit_price	discount_...	line_amount
AUSTRALIA	100069	AYLA	ADAMS	100000069	2021-03-17 00:00:00	10019	Rolltop Desk	108	Natural Ash	1	480.00	0.00	480.00000000
AUSTRALIA	100070	TALIA	ALLEN	100000070	2022-10-10 00:00:00	10020	Armoire	109	Oak	1	2100.00	30.00	1470.00000000
CHINA	100043	AI	Chen	100000043	2019-01-03 00:00:00	10018	Hair Dresser	102	Driftwood	3	845.00	5.00	2408.25000000
CHINA	100044	Bao	Cheng	100000044	2021-02-04 00:00:00	10019	Rolltop Desk	103	Fruitwood	1	480.00	10.00	432.00000000
CHINA	100045	Caihong	Ding	100000045	2018-01-05 00:00:00	10020	Armoire	104	Golden Oak	1	2100.00	15.00	1785.00000000
CHINA	100046	Chun	Deng	100000046	2021-05-06 00:00:00	10021	Windsor Chair	105	Green Velvet	2	340.00	19.00	550.80000000
CHINA	100047	Hua	Fang	100000047	2021-02-07 00:00:00	10022	Nightstand	106	Gunstock	3	230.00	45.00	379.50000000
CHINA	100048	Juan	Guo	100000048	2022-01-10 00:00:00	10023	TV Stand	107	Natural	6	550.00	20.00	2640.00000000
CHINA	100049	QIANG	Hong	100000049	2022-02-09 00:00:00	10024	Futon Bed	108	Natural Ash	5	1800.00	60.00	3600.00000000
CHINA	100050	WEI	Huang	100000050	2022-05-11 00:00:00	10025	School Chair	109	Oak	6	75.00	65.00	157.50000000
CHINA	100051	JIE	Jiang	100000051	2019-02-05 00:00:00	10001	Cradle Bed	110	Provincial	1	200.00	20.00	180.00000000
DENMARK	100080	Raymo	Meyer	100000110	2019-11-19 00:00:00	10005	Writer Desk	113	Simply White	1	350.00	10.00	315.00000000
DENMARK	100080	Raymo	Meyer	100000110	2019-11-19 00:00:00	10014	Futon Chair	114	Honey Maple	1	205.00	10.00	184.50000000
DENMARK	100080	Raymo	Meyer	100000110	2019-11-19 00:00:00	10021	Windsor Chair	109	Oak	2	340.00	45.00	374.00000000
DENMARK	100080	Raymo	Meyer	100000123	2021-03-17 00:00:00	10006	Settee	106	Gunstock	3	400.00	12.00	1056.00000000
DENMARK	100080	Raymo	Meyer	100000123	2021-03-17 00:00:00	10010	Height Bar S...	115	Dark Walnut	7	250.00	10.00	1575.00000000
DENMARK	100080	Raymo	Meyer	100000123	2021-03-17 00:00:00	10024	Futon Bed	107	Natural	1	1800.00	45.00	990.00000000
DENMARK	100080	Raymo	Meyer	100000136	2022-10-10 00:00:00	10004	Entertainme...	118	Vintage Aqua	1	4500.00	20.00	3600.00000000
DENMARK	100080	Raymo	Meyer	100000136	2022-10-10 00:00:00	10007	Dinning Table	103	Fruitwood	1	1100.00	30.00	770.00000000
DENMARK	100080	Raymo	Meyer	100000136	2022-10-10 00:00:00	10016	Washstand	110	Provincial	4	355.00	15.00	1207.00000000
DENMARK	100080	Raymo	Meyer	100000149	2022-07-13 00:00:00	10007	Dinning Table	103	Fruitwood	5	1100.00	60.00	2200.00000000
DENMARK	100080	Raymo	Meyer	100000149	2022-07-13 00:00:00	10018	Hair Dresser	100	Cherry	5	845.00	10.00	3802.50000000
DENMARK	100081	Juan	Parish	100000111	2021-03-17 00:00:00	10005	Writer Desk	114	Honey Maple	1	350.00	15.00	297.50000000
DENMARK	100081	Juan	Parish	100000111	2021-03-17 00:00:00	10014	Futon Chair	115	Dark Walnut	1	205.00	15.00	174.25000000
DENMARK	100081	Juan	Parish	100000111	2021-03-17 00:00:00	10021	Windsor Chair	110	Provincial	6	340.00	15.00	1734.00000000
DENMARK	100081	Juan	Parish	100000124	2022-10-10 00:00:00	10006	Settee	107	Natural	5	400.00	11.00	1790.00000000
DENMARK	100081	Juan	Parish	100000124	2022-10-10 00:00:00	10010	Height Bar S...	116	Black Cherry	4	250.00	50.00	500.00000000
DENMARK	100081	Juan	Parish	100000124	2022-10-10 00:00:00	10024	Futon Bed	108	Natural Ash	4	1800.00	15.00	6120.00000000
DENMARK	100081	Juan	Parish	100000137	2018-01-05 00:00:00	10004	Entertainme...	119	Spring Oak	2	4500.00	20.00	7200.00000000
DENMARK	100081	Juan	Parish	100000137	2018-01-05 00:00:00	10007	Dinning Table	104	Golden Oak	2	1100.00	10.00	1980.00000000
DENMARK	100081	Juan	Parish	100000137	2018-01-05 00:00:00	10016	Washstand	111	Puritan Pine	2	355.00	20.00	568.00000000
DENMARK	100081	Juan	Parish	100000150	2022-04-16 00:00:00	10002	Cabinets	101	Classic Gray	4	345.00	15.00	1173.00000000
DENMARK	100081	Juan	Parish	100000150	2022-04-16 00:00:00	10002	Cabinets	104	Golden Oak	4	345.00	20.00	1104.00000000
DENMARK	100081	Juan	Parish	100000150	2022-04-16 00:00:00	10007	Dinning Table	104	Golden Oak	6	1100.00	65.00	2310.00000000
DENMARK	100081	Juan	Parish	100000150	2022-04-16 00:00:00	10011	Dinning Chair	115	Dark Walnut	2	150.00	45.00	185.00000000
DENMARK	100081	Juan	Parish	100000150	2022-04-16 00:00:00	10018	Hair Dresser	101	Classic Gray	3	845.00	50.00	1267.50000000
DENMARK	100082	Jeffery	Coleman	100000112	2022-10-10 00:00:00	10005	Writer Desk	115	Dark Walnut	2	350.00	19.00	567.00000000
DENMARK	100082	Jeffery	Coleman	100000112	2022-10-10 00:00:00	10014	Futon Chair	116	Black Cherry	4	205.00	15.00	697.00000000
DENMARK	100082	Jeffery	Coleman	100000112	2022-10-10 00:00:00	10021	Windsor Chair	111	Puritan Pine	2	340.00	10.00	612.00000000
DENMARK	100082	Jeffery	Coleman	100000125	2018-01-05 00:00:00	10006	Settee	108	Natural Ash	7	400.00	8.00	2578.00000000
DENMARK	100082	Jeffery	Coleman	100000125	2018-01-05 00:00:00	10010	Height Bar S...	117	Burn Red	3	250.00	35.00	487.50000000
DENMARK	100082	Jeffery	Coleman	100000125	2018-01-05 00:00:00	10024	Futon Bed	109	Oak	3	1800.00	10.00	4860.00000000
DENMARK	100082	Jeffery	Coleman	100000138	2021-02-07 00:00:00	10004	Entertainme...	100	Cherry	1	4500.00	20.00	3600.00000000
DENMARK	100082	Jeffery	Coleman	100000138	2021-02-07 00:00:00	10007	Dinning Table	106	Dark Walnut	1	1100.00	5.00	1054.45000000

DENMARK	100082	Jeffery	Coleman	100000151	2019-06-18 00:00:00	10007	Dinning Table	105	Green Velvet	1	1100.00	20.00	880.00000000
DENMARK	100082	Jeffery	Coleman	100000151	2019-06-18 00:00:00	10018	Hair Dresser	102	Driftwood	1	845.00	15.00	718.25000000
DENMARK	100083	Amy	King	100000113	2018-01-05 00:00:00	10005	Writer Desk	116	Black Cherry	3	350.00	45.00	577.50000000
DENMARK	100083	Amy	King	100000113	2018-01-05 00:00:00	10014	Futon Chair	117	Barn Red	3	205.00	25.00	461.25000000
DENMARK	100083	Amy	King	100000113	2018-01-05 00:00:00	10021	Windsor Chair	112	Red Chestnut	5	340.00	50.00	850.00000000
DENMARK	100083	Amy	King	100000126	2021-02-07 00:00:00	10006	Settee	108	Natural Ash	4	400.00	5.00	1520.00000000
DENMARK	100083	Amy	King	100000126	2021-02-07 00:00:00	10020	Armoire	118	Vintage Aqua	1	2100.00	20.00	1680.00000000
DENMARK	100083	Amy	King	100000126	2021-02-07 00:00:00	10024	Futon Bed	110	Provincial	1	1800.00	50.00	900.00000000
DENMARK	100083	Amy	King	100000139	2022-02-09 00:00:00	10004	Entertainme...	101	Classic Gray	3	4500.00	45.00	7425.00000000
DENMARK	100083	Amy	King	100000139	2022-02-09 00:00:00	10007	Dinning Table	106	Gunstock	1	1100.00	20.00	880.00000000
DENMARK	100083	Amy	King	100000139	2022-02-09 00:00:00	10016	Washstand	113	Simply White	3	355.00	12.00	937.20000000
DENMARK	100083	Amy	King	100000152	2020-10-23 00:00:00	10002	Cabinets	103	Fruitwood	1	345.00	50.00	172.50000000
DENMARK	100083	Amy	King	100000152	2020-10-23 00:00:00	10007	Dinning Table	106	Gunstock	1	1100.00	21.00	869.00000000
DENMARK	100083	Amy	King	100000152	2020-10-23 00:00:00	10018	Hair Dresser	103	Fruitwood	5	845.00	20.00	3380.00000000
FRANCE	100068	Gilbert	Wolff	100000068	2019-11-19 00:00:00	10018	Hair Dresser	107	Natural	3	845.00	0.00	2535.00000000
GERMANY	100075	Jason	Hende...	100000105	2019-01-12 00:00:00	10005	Writer Desk	108	Natural Ash	3	350.00	12.00	924.00000000
GERMANY	100075	Jason	Hende...	100000105	2019-01-12 00:00:00	10014	Futon Chair	109	Oak	3	205.00	12.00	541.20000000
GERMANY	100075	Jason	Hende...	100000105	2019-01-12 00:00:00	10020	Armoire	104	Golden Oak	1	2100.00	45.00	1155.00000000
GERMANY	100075	Jason	Hende...	100000118	2019-05-17 00:00:00	10006	Settee	101	Classic Gray	1	400.00	21.00	316.00000000
GERMANY	100075	Jason	Hende...	100000118	2019-05-17 00:00:00	10010	Height Bar S...	110	Provincial	1	250.00	20.00	200.00000000
GERMANY	100075	Jason	Hende...	100000118	2019-05-17 00:00:00	10024	Futon Bed	102	Driftwood	1	1800.00	30.00	1260.00000000
GERMANY	100075	Jason	Hende...	100000131	2019-01-11 00:00:00	10004	Entertainme...	113	Simply White	5	4500.00	50.00	11250.00000000
GERMANY	100075	Jason	Hende...	100000131	2019-01-11 00:00:00	10007	Dinning Table	118	Vintage Aqua	3	1100.00	25.00	2475.00000000
GERMANY	100075	Jason	Hende...	100000131	2019-01-11 00:00:00	10016	Washstand	110	Provincial	5	355.00	10.00	1597.50000000
GERMANY	100075	Jason	Hende...	100000144	2021-10-15 00:00:00	10007	Dinning Table	118	Vintage Aqua	1	1100.00	10.00	990.00000000
GERMANY	100075	Jason	Hende...	100000144	2021-10-15 00:00:00	10018	Hair Dresser	106	Gunstock	1	845.00	20.00	676.00000000
GERMANY	100076	Jasmine	Gentry	100000106	2019-05-17 00:00:00	10005	Writer Desk	109	Oak	5	350.00	11.00	1557.50000000
GERMANY	100076	Jasmine	Gentry	100000106	2019-05-17 00:00:00	10014	Futon Chair	110	Provincial	5	205.00	11.00	912.25000000
GERMANY	100076	Jasmine	Gentry	100000106	2019-05-17 00:00:00	10021	Windsor Chair	105	Green Velvet	4	340.00	15.00	1156.00000000
GERMANY	100076	Jasmine	Gentry	100000119	2019-01-11 00:00:00	10006	Settee	102	Driftwood	1	400.00	23.00	308.00000000
GERMANY	100076	Jasmine	Gentry	100000119	2019-01-11 00:00:00	10010	Height Bar S...	111	Puritan Pine	2	250.00	20.00	400.00000000
GERMANY	100076	Jasmine	Gentry	100000119	2019-01-11 00:00:00	10024	Futon Bed	103	Fruitwood	2	1800.00	10.00	3240.00000000
GERMANY	100076	Jasmine	Gentry	100000132	2021-10-15 00:00:00	10004	Entertainme...	114	Honey Maple	3	4500.00	20.00	10800.000000...
GERMANY	100076	Jasmine	Gentry	100000132	2021-10-15 00:00:00	10007	Dinning Table	119	Spring Oak	2	1100.00	0.00	2200.00000000
GERMANY	100076	Jasmine	Gentry	100000132	2021-10-15 00:00:00	10016	Washstand	111	Puritan Pine	3	355.00	50.00	532.50000000
GERMANY	100076	Jasmine	Gentry	100000145	2021-04-17 00:00:00	10007	Dinning Table	119	Spring Oak	1	1100.00	15.00	935.00000000
GERMANY	100076	Jasmine	Gentry	100000145	2021-04-17 00:00:00	10018	Hair Dresser	107	Natural	1	845.00	15.00	718.25000000
GERMANY	100077	Caitlin	Torres	100000107	2019-01-11 00:00:00	10005	Writer Desk	110	Provincial	7	350.00	8.00	2254.00000000
GERMANY	100077	Caitlin	Torres	100000107	2019-01-11 00:00:00	10014	Futon Chair	111	Puritan Pine	7	205.00	8.00	1320.20000000
GERMANY	100077	Caitlin	Torres	100000107	2019-01-11 00:00:00	10021	Windsor Chair	106	Gunstock	3	340.00	10.00	918.00000000
GERMANY	100077	Caitlin	Torres	100000120	2021-10-15 00:00:00	10006	Settee	103	Fruitwood	1	400.00	0.00	400.00000000
GERMANY	100077	Caitlin	Torres	100000120	2021-10-15 00:00:00	10010	Height Bar S...	112	Red Chestnut	1	250.00	20.00	200.00000000
GERMANY	100077	Caitlin	Torres	100000120	2021-10-15 00:00:00	10024	Futon Bed	104	Golden Oak	1	1800.00	5.05	1709.10000000
GERMANY	100077	Caitlin	Torres	100000133	2021-04-17 00:00:00	10004	Entertainme...	115	Dark Walnut	1	4500.00	20.00	3600.00000000
GERMANY	100077	Caitlin	Torres	100000133	2021-04-17 00:00:00	10007	Dinning Table	100	Cherry	1	1100.00	0.00	1100.00000000
GERMANY	100077	Caitlin	Torres	100000133	2021-04-17 00:00:00	10016	Washstand	112	Red Chestnut	1	355.00	15.00	301.75000000
GERMANY	100077	Caitlin	Torres	100000146	2019-11-19 00:00:00	10007	Dinning Table	100	Cherry	2	1100.00	19.00	1782.00000000
GERMANY	100077	Caitlin	Torres	100000146	2019-11-19 00:00:00	10018	Hair Dresser	108	Natural Ash	4	845.00	20.00	2704.00000000
GERMANY	100078	Michael	Conner	100000108	2021-10-15 00:00:00	10005	Writer Desk	111	Puritan Pine	4	350.00	5.00	1330.00000000

country_name	cust_id	fname	lname	order_id	order_date	prod_id	prod_name	finish_id	finish_shade	quant	unit_price	discount_	line_amount
GERMANY	100078	Michael	Conner	100000108	2021-10-15 00:00:00	10014	Futon Chair	112	Red Chestnut	4	205.00	5.00	779.00000000
GERMANY	100078	Michael	Conner	100000108	2021-10-15 00:00:00	10021	Windsor Chair	107	Natural	1	340.00	50.00	170.00000000
GERMANY	100078	Michael	Conner	100000121	2021-04-17 00:00:00	10006	Settee	104	Golden Oak	2	400.00	0.00	800.00000000
GERMANY	100078	Michael	Conner	100000121	2021-04-17 00:00:00	10010	Height Bar S...	113	Simply White	3	250.00	45.00	412.50000000
GERMANY	100078	Michael	Conner	100000121	2021-04-17 00:00:00	10024	Futon Bed	105	Green Velvet	1	1800.00	20.00	1440.00000000
GERMANY	100078	Michael	Conner	100000134	2019-11-19 00:00:00	10004	Entertainment...	116	Black Cherry	1	4500.00	20.00	3600.00000000
GERMANY	100078	Michael	Conner	100000134	2019-11-19 00:00:00	10007	Dinning Table	101	Classic Gray	3	1100.00	0.00	3300.00000000
GERMANY	100078	Michael	Conner	100000134	2019-11-19 00:00:00	10016	Washstand	100	Cherry	5	355.00	20.00	1420.00000000
GERMANY	100078	Michael	Conner	100000147	2021-03-17 00:00:00	10007	Dinning Table	101	Classic Gray	3	1100.00	45.00	1815.00000000
GERMANY	100078	Michael	Conner	100000147	2021-03-17 00:00:00	10018	Hair Dresser	109	Oak	3	845.00	45.00	1394.25000000
GERMANY	100079	Joshua	Beil	100000109	2021-04-17 00:00:00	10005	Writer Desk	112	Red Chestnut	3	350.00	5.00	997.50000000
GERMANY	100079	Joshua	Beil	100000109	2021-04-17 00:00:00	10014	Futon Chair	113	Simply White	4	205.00	5.00	779.00000000
GERMANY	100079	Joshua	Beil	100000109	2021-04-17 00:00:00	10021	Windsor Chair	108	Natural Ash	4	340.00	20.00	1088.00000000
GERMANY	100079	Joshua	Beil	100000122	2019-11-19 00:00:00	10006	Settee	105	Green Velvet	1	400.00	0.00	400.00000000
GERMANY	100079	Joshua	Beil	100000122	2019-11-19 00:00:00	10010	Height Bar S...	114	Honey Maple	5	250.00	15.00	1062.50000000
GERMANY	100079	Joshua	Beil	100000122	2019-11-19 00:00:00	10024	Futon Bed	106	Gunstock	1	1800.00	30.00	1260.00000000
GERMANY	100079	Joshua	Beil	100000135	2021-03-17 00:00:00	10004	Entertainment...	117	Barn Red	1	4500.00	20.00	3600.00000000
GERMANY	100079	Joshua	Beil	100000135	2021-03-17 00:00:00	10007	Dinning Table	102	Driftwood	1	1100.00	0.00	1100.00000000
GERMANY	100079	Joshua	Beil	100000135	2021-03-17 00:00:00	10016	Washstand	109	Oak	1	355.00	12.00	312.40000000
GERMANY	100079	Joshua	Beil	100000148	2022-10-10 00:00:00	10007	Dinning Table	102	Driftwood	6	1100.00	20.00	5280.00000000
GERMANY	100079	Joshua	Beil	100000148	2022-10-10 00:00:00	10018	Hair Dresser	110	Provincial	6	845.00	15.00	4309.50000000
INDIA	100052	KARAN	MATH...	100000052	2019-01-12 00:00:00	10002	Cabinets	111	Puritan Pine	1	345.00	21.00	272.55000000
INDIA	100053	KIRAN	RAUT	100000053	2019-02-11 00:00:00	10003	Couch	112	Red Chestnut	1	3000.00	23.00	2310.00000000
INDIA	100054	CHAFU	SINGH	100000054	2019-05-17 00:00:00	10004	Entertainment...	113	Simply White	1	4500.00	0.00	4500.00000000
INDIA	100055	KUSHAL	RAO	100000055	2019-01-11 00:00:00	10005	Writer Desk	114	Honey Maple	2	350.00	0.00	700.00000000
INDIA	100056	ARVI	PATEL	100000056	2021-12-24 00:00:00	10006	Settee	115	Dark Walnut	1	400.00	0.00	400.00000000
INDIA	100057	VIPIN	GANG	100000057	2019-10-01 00:00:00	10007	Dinning Table	116	Black Cherry	3	1100.00	12.00	2904.00000000
INDIA	100058	SIMRAN	KALR	100000058	2018-12-02 00:00:00	10008	Coffee Table	117	Barn Red	5	370.00	11.00	1648.50000000
INDIA	100059	VISHAL	RAVAT	100000059	2018-01-05 00:00:00	10009	Stool	118	Vintage Aqua	7	100.00	8.00	644.00000000
INDIA	100060	DISHA	KHURJ...	100000060	2021-02-07 00:00:00	10010	Height Bar S...	119	Spring Oak	4	250.00	5.00	950.00000000
INDIA	100061	Chand...	Chaudhri	100000061	2022-02-09 00:00:00	10011	Dinning Chair	100	Cherry	4	150.00	5.00	570.00000000
INDIA	100062	AMRIT	RAO	100000062	2019-02-05 00:00:00	10012	Accent Chair	101	Classic Gray	1	425.00	10.00	382.50000000
INDIA	100063	TARAL	JANA	100000063	2019-01-12 00:00:00	10013	Cafe Chair	102	Driftwood	1	175.00	15.00	148.75000000
INDIA	100064	KAJAL	SINGH	100000064	2019-05-17 00:00:00	10014	Futon Chair	103	Fruitwood	4	205.00	15.00	697.00000000
INDIA	100065	HARSH	KHAN...	100000065	2019-01-11 00:00:00	10015	Wardrobe	104	Golden Oak	3	2500.00	25.00	5625.00000000
ITALY	100066	Gavino	Sove	100000066	2021-10-15 00:00:00	10016	Washstand	105	Green Velvet	2	355.00	0.00	710.00000000
ITALY	100067	Angela	Jephson	100000067	2021-04-17 00:00:00	10017	Bookcase	106	Gunstock	1	650.00	0.00	650.00000000
SWITZERLA...	100071	Charles	Patel	100000101	2018-01-05 00:00:00	10007	Dinning Table	107	Natural	1	1100.00	23.00	847.00000000
SWITZERLA...	100071	Charles	Patel	100000101	2018-01-05 00:00:00	10018	Hair Dresser	104	Golden Oak	1	845.00	12.00	743.60000000
SWITZERLA...	100071	Charles	Patel	100000101	2018-01-05 00:00:00	10020	Armoire	100	Cherry	2	2100.00	10.00	3780.00000000
SWITZERLA...	100071	Charles	Patel	100000114	2021-02-07 00:00:00	10005	Writer Desk	117	Barn Red	6	350.00	20.00	1980.00000000
SWITZERLA...	100071	Charles	Patel	100000114	2021-02-07 00:00:00	10014	Futon Chair	118	Vintage Aqua	2	205.00	0.00	410.00000000
SWITZERLA...	100071	Charles	Patel	100000114	2021-02-07 00:00:00	10021	Windsor Chair	113	Simply White	3	340.00	20.00	916.00000000
SWITZERLA...	100071	Charles	Patel	100000127	2022-02-09 00:00:00	10006	Settee	114	Honey Maple	4	400.00	5.00	1520.00000000
SWITZERLA...	100071	Charles	Patel	100000127	2022-02-09 00:00:00	10020	Armoire	119	Spring Oak	1	2100.00	15.00	1785.00000000
SWITZERLA...	100071	Charles	Patel	100000127	2022-02-09 00:00:00	10024	Futon Bed	109	Oak	4	1800.00	20.00	5760.00000000
SWITZERLA...	100071	Charles	Patel	100000140	2019-02-05 00:00:00	10004	Entertainment...	102	Driftwood	5	4500.00	15.00	19125.000000...
SWITZERLA...	100071	Charles	Patel	100000140	2019-02-05 00:00:00	10007	Dinning Table	114	Honey Maple	5	1100.00	11.00	4895.00000000
SWITZERLA...	100071	Charles	Patel	100000140	2019-02-05 00:00:00	10007	Dinning Table	107	Natural	1	1100.00	30.00	770.00000000

country_name	cust_id	fname	lname	order_id	order_date	prod_id	prod_name	finish_id	finish_shade	quant	unit_price	discount_	line_amount
SWITZERLA...	100071	Charles	Patel	100000140	2019-02-05 00:00:00	10007	Dinning Table	107	Natural	1	1100.00	30.00	770.00000000
SWITZERLA...	100072	Daniel	Franklin	100000102	2021-02-07 00:00:00	10014	Futon Chair	108	Natural Ash	1	205.00	0.00	205.00000000
SWITZERLA...	100072	Daniel	Franklin	100000102	2021-02-07 00:00:00	10018	Hair Dresser	105	Green Velvet	4	845.00	15.00	2873.00000000
SWITZERLA...	100072	Daniel	Franklin	100000102	2021-02-07 00:00:00	10020	Armoire	101	Classic Gray	1	2100.00	5.00	1993.90000000
SWITZERLA...	100072	Daniel	Franklin	100000115	2022-02-09 00:00:00	10005	Writer Desk	118	Vintage Aqua	5	350.00	80.00	700.00000000
SWITZERLA...	100072	Daniel	Franklin	100000115	2022-02-09 00:00:00	10014	Futon Chair	119	Spring Oak	1	205.00	0.00	205.00000000
SWITZERLA...	100072	Daniel	Franklin	100000115	2022-02-09 00:00:00	10021	Windsor Chair	114	Honey Maple	1	340.00	20.00	272.00000000
SWITZERLA...	100072	Daniel	Franklin	100000128	2019-02-05 00:00:00	10006	Settee	115	Dark Walnut	1	400.00	10.00	380.00000000
SWITZERLA...	100072	Daniel	Franklin	100000128	2019-02-05 00:00:00	10020	Armoire	100	Cherry	4	2100.00	20.00	6720.00000000
SWITZERLA...	100072	Daniel	Franklin	100000128	2019-02-05 00:00:00	10024	Futon Bed	110	Provincial	2	1800.00	45.00	1980.00000000
SWITZERLA...	100072	Daniel	Franklin	100000141	2019-01-12 00:00:00	10007	Dinning Table	115	Dark Walnut	7	1100.00	8.00	7084.00000000
SWITZERLA...	100072	Daniel	Franklin	100000141	2019-01-12 00:00:00	10007	Dinning Table	108	Natural Ash	1	1100.00	45.00	605.00000000
SWITZERLA...	100072	Daniel	Franklin	100000141	2019-01-12 00:00:00	10018	Hair Dresser	103	Fruitwood	7	845.00	10.00	5323.00000000
SWITZERLA...	100073	Jonathan	Greer	100000103	2022-02-09 00:00:00	10005	Writer Desk	106	Gunstock	2	350.00	20.00	580.00000000
SWITZERLA...	100073	Jonathan	Greer	100000103	2022-02-09 00:00:00	10014	Futon Chair	109	Oak	2	205.00	0.00	410.00000000
SWITZERLA...	100073	Jonathan	Greer	100000103	2022-02-09 00:00:00	10020	Armoire	102	Driftwood	1	2100.00	20.00	1980.00000000
SWITZERLA...	100073	Jonathan	Greer	100000116	2019-02-05 00:00:00	10006	Settee	119	Spring Oak	6	400.00	65.00	840.00000000
SWITZERLA...	100073	Jonathan	Greer	100000116	2019-02-05 00:00:00	10010	Height Bar S...	115	Dark Walnut	1	250.00	20.00	200.00000000
SWITZERLA...	100073	Jonathan	Greer	100000116	2019-02-05 00:00:00	10014	Futon Chair	100	Cherry	3	205.00	0.00	615.00000000
SWITZERLA...	100073	Jonathan	Greer	100000129	2019-01-12 00:00:00	10006	Settee	118	Black Cherry	1	400.00	15.00	340.00000000
SWITZERLA...	100073	Jonathan	Greer	100000129	2019-01-12 00:00:00	10018	Washstand	101	Classic Gray	3	355.00	45.00	585.75000000
SWITZERLA...	100073	Jonathan	Greer	100000129	2019-01-12 00:00:00	10024	Futon Bed	111	Puritan Pine	6	1800.00	15.00	9180.00000000
SWITZERLA...	100073	Jonathan	Greer	100000142	2019-05-17 00:00:00	10007	Dinning Table	116	Black Cherry	4	1100.00	5.00	4180.00000000
SWITZERLA...	100073	Jonathan	Greer	100000142	2019-05-17 00:00:00	10018	Hair Dresser	104	Golden Oak	4	845.00	50.00	1690.00000000
SWITZERLA...	100074	Cristina	Garcia	100000104	2019-02-05 00:00:00	10005	Writer Desk	107	Natural	1	350.00	30.00	245.00000000
SWITZERLA...	100074	Cristina	Garcia	100000104	2019-02-05 00:00:00	10014	Futon Chair	110	Provincial	1	205.00	0.00	205.00000000
SWITZERLA...	100074	Cristina	Garcia	100000104	2019-02-05 00:00:00	10020	Armoire	103	Fruitwood	1	2100.00	30.00	1470.00000000
SWITZERLA...	100074	Cristina	Garcia	100000117	2019-01-12 00:00:00	10006	Settee	100	Cherry	1	400.00	20.00	320.00000000
SWITZERLA...	100074	Cristina	Garcia	100000117	2019-01-12 00:00:00	10010	Height Bar S...	102	Driftwood	1	250.00	20.00	200.00000000
SWITZERLA...	100074	Cristina	Garcia	100000117	2019-01-12 00:00:00	10014	Futon Chair	101	Classic Gray	1	205.00	0.00	205.00000000
SWITZERLA...	100074	Cristina	Garcia	100000130	2019-05-17 00:00:00	10006	Settee	117	Barn Red	4	400.00	15.00	1360.00000000
SWITZERLA...	100074	Cristina	Garcia	100000130	2019-05-17 00:00:00	10018	Washstand	102	Driftwood	6	355.00	15.00	1810.50000000
SWITZERLA...	100074	Cristina	Garcia	100000130	2019-05-17 00:00:00	10024	Futon Bed	112	Red Chestnut	2	1800.00	10.00	3240.00000000
SWITZERLA...	100074	Cristina	Garcia	100000143	2019-01-11 00:00:00	10007	Dinning Table	117	Barn Red	3	1100.00	5.00	3135.00000000
SWITZERLA...	100074	Cristina	Garcia	100000143	2019-01-11 00:00:00	10018	Hair Dresser	105	Green Velvet	3	845.00	35.00	1647.75000000
UK	100031	STEVE	HALL	100000031	2021-12-16 00:00:00	10006	Settee	110	Provincial	5	400.00	10.00	1800.00000000
UK	100032	Mina	Nylund	100000032	2021-09-18 00:00:00	10007	Dinning Table	111	Puritan Pine	3	1100.00	50.00	1650.00000000
UK	100033	Kari	Anzly	100000033	2022-02-19 00:00:00	10006	Coffee Table	112	Red Chestnut	1	370.00	15.00	314.50000000
UK	100034	Cassu	Peymrod	100000034	2019-10-13 00:00:00	10008	Stool	113	Simply White	5	100.00	20.00	400.00000000
UK	100035	RUSS...	RYDER	100000035	2022-09-15 00:00:00	10010	Height Bar S...	114	Honey Maple	1	250.00	12.00	220.00000000
UK	100036	BOBBY	PARKER	100000036	2021-12-17 00:00:00	10011	Dinning Chair	115	Dark Walnut	4	150.00	15.00	510.00000000
UK	100037	VICTOR	TURN...	100000037	2019-10-19 00:00:00	10012	Accent Chair	116	Black Cherry	2	425.00	20.00	680.00000000
UK	100038	ERNEST	COOK	100000038	2021-09-25 00:00:00	10013	Cafe Chair	117	Barn Red	1	175.00	30.00	122.50000000
UK	100039	PHILLIP	COOP...	100000039	2021-12-24 00:00:00	10014	Futon Chair	118	Vintage Aqua	3	205.00	12.00	541.20000000
UK	100040	ALAN	BAILEY	100000040	2021-09-16 00:00:00	10015	Wardrobe	119	Spring Oak	5	2500.00	11.00	11125.00000000
UK	100041	SHAWN	REED	100000041	2019-10-01 00:00:00	10016	Washstand	100	Cherry	7	355.00	8.00	2286.20000000
UK	100042	SEAN	COX	100000042	2018-12-02 00:00:00	10017	Bookcase	101	Classic Gray	4	660.00	5.00	2470.00000000
US	100001	MIKE	Ashton	100000001	2021-06-11 00:00:00	10001	Cradle Bed	100	Cherry	2	200.00	10.00	360.00000000
US	100001	MIKE	Ashton	100000001	2021-06-11 00:00:00	10025	School Chair	101	Classic Gray	4	75.00	13.00	261.00000000

US	100001	MIKE	Ashton	100000001	2021-06-11 00:00:00	10025	School Chair	119	Spring Oak	5	75.00	35.00	243.75000000
US	100001	MIKE	Ashton	100000071	2021-04-28 00:00:00	10021	Windsor Chair	110	Provincial	2	340.00	31.00	408.20000000
US	100002	LUCY	Baron	100000002	2019-07-12 00:00:00	10001	Cradle Bed	100	Cherry	2	200.00	5.00	380.00000000
US	100002	LUCY	Baron	100000002	2019-07-12 00:00:00	10002	Cabinets	101	Classic Gray	1	345.00	5.05	327.57500000
US	100002	LUCY	Baron	100000002	2019-07-12 00:00:00	10009	Stool	102	Driftwood	1	100.00	18.00	82.00000000
US	100002	LUCY	Baron	100000072	2021-06-11 00:00:00	10022	Nightstand	111	Puritan Pine	1	230.00	40.00	138.00000000
US	100003	MAKA	Baron	100000003	2022-04-13 00:00:00	10002	Cabinets	101	Classic Gray	1	345.00	10.00	310.50000000
US	100003	MAKA	Baron	100000003	2022-04-13 00:00:00	10003	Couch	102	Driftwood	1	3000.00	20.00	2400.00000000
US	100003	MAKA	Baron	100000003	2022-04-13 00:00:00	10010	Height Bar S...	110	Provincial	2	250.00	19.00	405.00000000
US	100003	MAKA	Baron	100000003	2022-04-13 00:00:00	10019	Holtop Desk	115	Dark Walnut	3	480.00	5.00	1368.00000000
US	100003	MAKA	Baron	100000073	2019-07-12 00:00:00	10023	TV Stand	112	Red Chestnut	2	550.00	50.00	550.00000000
US	100004	KELLY	Canon	100000004	2022-03-14 00:00:00	10003	Couch	102	Driftwood	1	3000.00	10.00	2700.00000000
US	100004	KELLY	Canon	100000004	2022-03-14 00:00:00	10004	Entertainme...	103	Fruitwood	1	4500.00	30.00	3150.00000000
US	100004	KELLY	Canon	100000004	2022-03-14 00:00:00	10011	Dinning Chair	111	Puritan Pine	3	150.00	20.00	360.00000000
US	100004	KELLY	Canon	100000004	2022-03-14 00:00:00	10020	Armoire	116	Black Cherry	1	2100.00	75.00	525.00000000
US	100004	KELLY	Canon	100000074	2021-10-15 00:00:00	10024	Futon Bed	113	Simply White	1	1800.00	50.00	900.00000000
US	100005	JAMIE	Finley	100000005	2021-10-15 00:00:00	10004	Entertainme...	103	Fruitwood	1	4500.00	21.00	3555.00000000
US	100005	JAMIE	Finley	100000005	2021-10-15 00:00:00	10005	Writer Desk	104	Golden Oak	1	350.00	45.00	192.50000000
US	100005	JAMIE	Finley	100000005	2021-10-15 00:00:00	10012	Accent Chair	112	Red Chestnut	2	425.00	45.00	467.50000000
US	100005	JAMIE	Finley	100000005	2021-10-15 00:00:00	10021	Windsor Chair	117	Barn Red	1	340.00	70.00	102.00000000
US	100005	JAMIE	Finley	100000075	2021-03-17 00:00:00	10025	School Chair	114	Honey Maple	3	75.00	5.00	213.75000000
US	100006	NICK	Finley	100000006	2019-11-19 00:00:00	10005	Writer Desk	104	Golden Oak	1	350.00	15.00	297.50000000
US	100006	NICK	Finley	100000006	2019-11-19 00:00:00	10006	Settee	105	Green Velvet	4	400.00	15.00	1360.00000000
US	100006	NICK	Finley	100000006	2019-11-19 00:00:00	10022	Nightstand	118	Vintage Aqua	4	230.00	60.00	368.00000000
US	100006	NICK	Finley	100000076	2022-10-10 00:00:00	10001	Cradle Bed	115	Dark Walnut	1	200.00	5.00	190.00000000
US	100007	HARRY	Hanson	100000007	2021-03-17 00:00:00	10006	Settee	105	Green Velvet	1	400.00	20.00	320.00000000
US	100007	HARRY	Hanson	100000007	2021-03-17 00:00:00	10007	Dinning Table	106	Gunstock	3	1100.00	10.00	2970.00000000
US	100007	HARRY	Hanson	100000007	2021-03-17 00:00:00	10023	TV Stand	119	Spring Oak	2	550.00	0.00	1100.00000000
US	100007	HARRY	Hanson	100000077	2015-04-19 00:00:00	10002	Cabinets	118	Black Cherry	7	345.00	5.00	2294.25000000
US	100008	Derrick	Snyders	100000008	2022-07-18 00:00:00	10007	Dinning Table	106	Gunstock	1	1100.00	30.00	770.00000000
US	100008	Derrick	Snyders	100000008	2022-07-18 00:00:00	10008	Coffee Table	107	Natural	1	370.00	50.00	185.00000000
US	100008	Derrick	Snyders	100000078	2020-10-23 00:00:00	10003	Couch	117	Barn Red	4	3050.00	0.00	12000.00000000
US	100009	Lon	Bonher	100000009	2022-10-10 00:00:00	10008	Coffee Table	107	Natural	1	370.00	35.00	240.50000000
US	100009	Lon	Bonher	100000009	2022-10-10 00:00:00	10009	Stool	108	Natural Ash	4	100.00	20.00	320.00000000
US	100009	Lon	Bonher	100000079	2022-04-13 00:00:00	10004	Entertainme...	118	Vintage Aqua	3	4500.00	0.00	13500.00000000
US	100010	JAMES SMITH		100000010	2021-04-28 00:00:00	10008	Stool	108	Natural Ash	4	100.00	40.00	240.00000000
US	100010	JAMES SMITH		100000010	2021-04-28 00:00:00	10010	Height Bar S...	109	Oak	2	250.00	45.00	275.00000000
US	100010	JAMES SMITH		100000080	2021-09-18 00:00:00	10005	Writer Desk	119	Spring Oak	1	350.00	20.00	280.00000000
US	100011	Flannery	Newton	100000011	2019-03-10 00:00:00	10010	Height Bar S...	109	Oak	1	250.00	27.00	182.50000000
US	100011	Flannery	Newton	100000011	2019-03-10 00:00:00	10011	Dinning Chair	110	Provincial	6	150.00	15.00	765.00000000
US	100011	Flannery	Newton	100000081	2019-10-13 00:00:00	10006	Settee	100	Cherry	1	400.00	17.00	332.00000000
US	100012	ROBERT JONES		100000012	2018-07-14 00:00:00	10005	Writer Desk	114	Honey Maple	3	350.00	21.00	829.50000000
US	100012	ROBERT JONES		100000012	2018-07-14 00:00:00	10011	Dinning Chair	110	Provincial	6	150.00	15.00	765.00000000
US	100012	ROBERT JONES		100000082	2021-12-17 00:00:00	10012	Accent Chair	111	Puritan Pine	2	425.00	10.00	765.00000000
US	100013	RICHARD MILLER		100000013	2019-07-18 00:00:00	10007	Dinning Table	101	Classic Gray	4	1100.00	20.00	3520.00000000
US	100013	RICHARD MILLER		100000013	2019-07-18 00:00:00	10006	Settee	115	Dark Walnut	2	400.00	25.00	600.00000000
US	100013	RICHARD MILLER		100000013	2019-07-18 00:00:00	10012	Accent Chair	111	Puritan Pine	4	425.00	5.00	1615.00000000
US	100013	RICHARD MILLER		100000013	2019-07-18 00:00:00	10013	Cafe Chair	112	Red Chestnut	5	175.00	50.00	437.50000000
US	100013	RICHARD MILLER		100000083	2021-09-25 00:00:00	10008	Coffee Table	102	Driftwood	3	370.00	25.00	832.50000000

country_name	cust_id	fname	lname	order_id	order_date	prod_id	prod_name	finish_id	finish_shade	quant	unit_price	discount_	line_amount
US	100013	RICHA	MILLER	100000083	2021-09-25 00:00:00	10008	Coffee Table	102	Driftwood	3	370.00	25.00	832.50000000
US	100014	JOSEPH	DAVIS	100000014	2015-04-19 00:00:00	10007	Dinning Table	116	Black Cherry	1	1100.00	13.00	957.00000000
US	100014	JOSEPH	DAVIS	100000014	2015-04-19 00:00:00	10013	Cafe Chair	112	Red Chestnut	1	175.00	5.00	166.25000000
US	100014	JOSEPH	DAVIS	100000014	2015-04-19 00:00:00	10014	Futon Chair	113	Simply White	3	205.00	20.00	492.00000000
US	100014	JOSEPH	DAVIS	100000084	2021-02-04 00:00:00	10009	Stool	103	Fruitwood	6	100.00	30.00	420.00000000
US	100015	THOM...	WILSON	100000015	2018-10-19 00:00:00	10008	Coffee Table	117	Barn Red	3	370.00	25.00	832.50000000
US	100015	THOM...	WILSON	100000015	2018-10-19 00:00:00	10014	Futon Chair	113	Simply White	3	205.00	16.00	516.60000000
US	100015	THOM...	WILSON	100000015	2018-10-19 00:00:00	10015	Wardrobe	114	Honey Maple	1	2500.00	20.00	2000.00000000
US	100015	THOM...	WILSON	100000085	2021-02-07 00:00:00	10010	Height Bar S...	104	Golden Oak	5	250.00	35.00	812.50000000
US	100016	CHRIS...	MARTIN	100000016	2019-12-17 00:00:00	10009	Stool	118	Vintage Aqua	1	100.00	35.00	65.00000000
US	100016	CHRIS...	MARTIN	100000016	2019-12-17 00:00:00	10015	Wardrobe	114	Honey Maple	2	2500.00	20.00	4000.00000000
US	100016	CHRIS...	MARTIN	100000016	2019-12-17 00:00:00	10016	Washstand	115	Dark Walnut	1	355.00	20.00	284.00000000
US	100016	CHRIS...	MARTIN	100000086	2022-05-11 00:00:00	10011	Dinning Chair	105	Green Velvet	8	150.00	5.00	1140.00000000
US	100017	MARK	THOM...	100000017	2021-04-14 00:00:00	10010	Height Bar S...	119	Spring Oak	1	250.00	10.00	225.00000000
US	100017	MARK	THOM...	100000017	2021-04-14 00:00:00	10016	Washstand	115	Dark Walnut	1	355.00	52.00	170.40000000
US	100017	MARK	THOM...	100000017	2021-04-14 00:00:00	10017	Bookcase	118	Black Cherry	1	650.00	20.00	520.00000000
US	100017	MARK	THOM...	100000087	2019-02-06 00:00:00	10012	Accent Chair	106	Gunstock	2	425.00	0.00	850.00000000
US	100018	DONALD	TRUMP	100000018	2021-11-12 00:00:00	10011	Dinning Chair	100	Cherry	2	150.00	5.00	285.00000000
US	100018	DONALD	TRUMP	100000018	2021-11-12 00:00:00	10017	Bookcase	116	Black Cherry	2	650.00	27.00	949.00000000
US	100018	DONALD	TRUMP	100000018	2021-11-12 00:00:00	10018	Hair Dresser	117	Barn Red	1	845.00	20.00	676.00000000
US	100018	DONALD	TRUMP	100000088	2022-04-13 00:00:00	10013	Cafe Chair	107	Natural	1	175.00	10.00	157.50000000
US	100019	BRIAN	HARRIS	100000019	2022-07-13 00:00:00	10001	Cradle Bed	100	Cherry	1	200.00	0.00	200.00000000
US	100019	BRIAN	HARRIS	100000019	2022-07-13 00:00:00	10012	Accent Chair	101	Classic Gray	1	425.00	75.00	106.25000000
US	100019	BRIAN	HARRIS	100000019	2022-07-13 00:00:00	10018	Hair Dresser	117	Barn Red	1	845.00	15.00	718.25000000
US	100019	BRIAN	HARRIS	100000019	2022-07-13 00:00:00	10019	Rolltop Desk	118	Vintage Aqua	2	480.00	20.00	768.00000000
US	100019	BRIAN	HARRIS	100000089	2018-07-14 00:00:00	10014	Futon Chair	108	Natural Ash	1	205.00	0.00	205.00000000
US	100020	ANTH...	CLARK	100000020	2022-04-16 00:00:00	10013	Cafe Chair	102	Driftwood	2	175.00	21.00	276.50000000
US	100020	ANTH...	CLARK	100000020	2022-04-16 00:00:00	10019	Rolltop Desk	118	Vintage Aqua	1	480.00	5.00	456.00000000
US	100020	ANTH...	CLARK	100000020	2022-04-16 00:00:00	10020	Armoire	119	Spring Oak	1	2100.00	20.00	1680.00000000
US	100020	ANTH...	CLARK	100000090	2019-01-11 00:00:00	10015	Wardrobe	109	Oak	1	2500.00	0.00	2500.00000000
US	100021	KEVIN	LEWIS	100000021	2019-06-18 00:00:00	10014	Futon Chair	110	Provincial	1	205.00	30.00	143.50000000
US	100021	KEVIN	LEWIS	100000021	2019-06-18 00:00:00	10020	Armoire	119	Spring Oak	2	2100.00	5.00	3990.00000000
US	100021	KEVIN	LEWIS	100000021	2019-06-18 00:00:00	10021	Windsor Chair	100	Cherry	3	340.00	45.00	561.00000000
US	100021	KEVIN	LEWIS	100000091	2022-05-11 00:00:00	10016	Washstand	110	Provincial	2	355.00	0.00	710.00000000
US	100022	GARY	WALK...	100000022	2020-10-23 00:00:00	10001	Cradle Bed	110	Provincial	4	200.00	5.00	790.00000000
US	100022	GARY	WALK...	100000022	2020-10-23 00:00:00	10015	Wardrobe	111	Puritan Pine	3	2500.00	13.00	6525.00000000
US	100022	GARY	WALK...	100000022	2020-10-23 00:00:00	10022	Nightstand	101	Classic Gray	5	230.00	15.00	977.50000000
US	100022	GARY	WALK...	100000092	2019-05-16 00:00:00	10017	Bookcase	111	Puritan Pine	1	650.00	9.00	591.50000000
US	100023	JOSE	SANC...	100000023	2018-07-24 00:00:00	10002	Cabinets	111	Puritan Pine	1	345.00	10.00	310.50000000
US	100023	JOSE	SANC...	100000023	2018-07-24 00:00:00	10016	Washstand	112	Red Chestnut	1	355.00	25.00	266.25000000
US	100023	JOSE	SANC...	100000023	2018-07-24 00:00:00	10023	TV Stand	102	Driftwood	7	550.00	10.00	3465.00000000
US	100023	JOSE	SANC...	100000093	2022-07-18 00:00:00	10018	Hair Dresser	112	Red Chestnut	1	845.00	8.00	777.40000000
US	100024	SCOTT	KING	100000024	2017-03-11 00:00:00	10003	Couch	112	Red Chestnut	1	3000.00	5.00	2850.00000000
US	100024	SCOTT	KING	100000024	2017-03-11 00:00:00	10017	Bookcase	113	Simply White	7	650.00	35.00	2957.50000000
US	100024	SCOTT	KING	100000024	2017-03-11 00:00:00	10024	Futon Bed	103	Fruitwood	4	1800.00	50.00	3600.00000000
US	100024	SCOTT	KING	100000094	2021-03-17 00:00:00	10019	Rolltop Desk	113	Simply White	2	480.00	6.00	902.40000000
US	100025	ERIC	WRIGHT	100000025	2021-06-13 00:00:00	10004	Entertainme...	113	Simply White	4	4500.00	75.00	4500.00000000
US	100025	ERIC	WRIGHT	100000025	2021-06-13 00:00:00	10018	Hair Dresser	114	Honey Maple	4	845.00	10.00	3042.00000000
US	100025	ERIC	WRIGHT	100000025	2021-06-13 00:00:00	10025	School Chair	104	Golden Oak	3	75.00	35.00	146.25000000
US	100025	ERIC	WRIGHT	100000025	2021-06-13 00:00:00	10025	School Chair	104	Golden Oak	3	75.00	35.00	146.25000000
US	100025	ERIC	WRIGHT	100000095	2019-11-19 00:00:00	10020	Armoire	114	Honey Maple	1	2100.00	5.00	1995.00000000
US	100026	WILLIE	HILL	100000026	2021-07-16 00:00:00	10001	Cradle Bed	105	Green Velvet	1	200.00	20.00	160.00000000
US	100026	WILLIE	HILL	100000096	2021-05-06 00:00:00	10021	Windsor Chair	115	Dark Walnut	2	340.00	5.00	648.00000000
US	100027	NICH...	FLORES	100000027	2021-10-17 00:00:00	10002	Cabinets	106	Gunstock	1	345.00	15.00	293.25000000
US	100027	NICH...	FLORES	100000097	2021-09-16 00:00:00	10022	Nightstand	116	Black Cherry	1	230.00	5.00	218.50000000
US	100028	BRUCE	ADAMS	100000028	2022-06-18 00:00:00	10003	Couch	107	Natural	4	3000.00	20.00	9600.00000000
US	100028	BRUCE	ADAMS	100000098	2021-12-17 00:00:00	10023	TV Stand	117	Barn Red	2	550.00	5.00	1045.00000000
US	100029	FRED	CAMP...	100000029	2022-06-19 00:00:00	10004	Entertainme...	108	Natural Ash	3	4500.00	45.00	7425.00000000
US	100029	FRED	CAMP...	100000099	2021-10-17 00:00:00	10024	Futon Bed	100	Cherry	3	1800.00	20.00	4320.00000000
US	100029	FRED	CAMP...	100000099	2021-10-17 00:00:00	10024	Futon Bed	118	Vintage Aqua	2	1800.00	30.00	2520.00000000
US	100030	HOWA...	CART...	100000030	2022-04-13 00:00:00	10005	Writer Desk	109	Oak	6	350.00	15.00	1785.00000000

2.1.3 Table join3

Business question:

For each country and product, show total sales amount.

SQL:

WHW:

- How: Join country, customer, order, order line and product tables and aggregate by country + product.

- Why: To compare product performance across different countries.

```
SELECT
  co.country_name,
  p.prod_id,
  p.name AS prod_name,
  SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) AS total_sales
FROM gb_country co
JOIN gb_cust c ON co.country_id = c.country_id
JOIN gb_order o ON c.cust_id = o.cust_id
JOIN gb_ord_prod op ON o.order_id = op.order_id
JOIN gb_prod p ON op.prod_id = p.prod_id
GROUP BY
  co.country_name,
  p.prod_id,
  p.name
ORDER BY
  co.country_name,
  total_sales DESC;
```

Result screenshot:

The screenshot displays a SQL IDE interface. The top pane shows a SQL query with line numbers 65 to 90. The query is a SELECT statement with JOINs and a GROUP BY clause. The bottom pane shows the execution output, including a table with columns: Time, Action, and Response. The response indicates that 100 rows were returned.

```
65 --
66 -- SQL JOIN 3
67 --
68 -- What! For each country and product, show total sales amount.
69 -- How! Join country, customer, order, order line and product tables and aggregate by country + product.
70 -- Why! To compare product performance across different countries.
71 --
72 SELECT
73   co.country_name,
74   p.prod_id,
75   p.name AS prod_name,
76   SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) AS total_sales
77 FROM gb_country co
78 JOIN gb_cust c ON co.country_id = c.country_id
79 JOIN gb_order o ON c.cust_id = o.cust_id
80 JOIN gb_ord_prod op ON o.order_id = op.order_id
81 JOIN gb_prod p ON op.prod_id = p.prod_id
82 GROUP BY
83   co.country_name,
84   p.prod_id,
85   p.name
86 ORDER BY
87   co.country_name,
88   total_sales DESC;
89
90
```

Time	Action	Response
19:55:23	SELECT co.country_name, p.prod_id, p.name AS prod_name, SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) AS total_sales	100 row(s) returned

	country_name	prod_id	prod_name	total_sales	
	INDIA	10009	Stool	644.00000000	
	INDIA	10011	Dinning Chair	570.00000000	
	INDIA	10006	Settee	400.00000000	
	INDIA	10012	Accent Chair	382.50000000	
	INDIA	10002	Cabinets	272.55000000	
	INDIA	10013	Cafe Chair	148.75000000	
	ITALY	10016	Washstand	710.00000000	
	ITALY	10017	Bookcase	650.00000000	
	SWITZERLA...	10007	Dinning Table	21516.000000...	
	SWITZERLA...	10024	Futon Bed	20160.000000...	
	SWITZERLA...	10004	Entertainme...	19125.000000...	
	SWITZERLA...	10020	Armoire	17428.950000...	
	SWITZERLA...	10018	Hair Dresser	12277.850000...	
	SWITZERLA...	10006	Settee	4740.00000000	
	SWITZERLA...	10005	Writer Desk	3185.00000000	
	SWITZERLA...	10016	Washstand	2396.25000000	
	SWITZERLA...	10014	Futon Chair	2255.00000000	
	SWITZERLA...	10021	Windsor Chair	1088.00000000	
	SWITZERLA...	10010	Height Bar S...	400.00000000	
	UK	10015	Wardrobe	11125.00000000	
	UK	10017	Bookcase	2470.00000000	
	UK	10016	Washstand	2286.20000000	
	UK	10006	Settee	1800.00000000	
	UK	10007	Dinning Table	1650.00000000	
	UK	10012	Accent Chair	680.00000000	
	UK	10014	Futon Chair	541.20000000	
	UK	10011	Dinning Chair	510.00000000	
	UK	10009	Stool	400.00000000	
	UK	10008	Coffee Table	314.50000000	
	UK	10010	Height Bar S...	220.00000000	
	UK	10013	Cafe Chair	122.50000000	
	US	10004	Entertainme...	32130.000000...	
	US	10003	Couch	29550.000000...	
	US	10015	Wardrobe	15025.000000...	
	US	10024	Futon Bed	11340.00000000	
	US	10007	Dinning Table	8217.00000000	
	US	10009	Stool	644.00000000	

country_name	prod_id	prod_name	total_sales	
AUSTRALIA	10020	Armoire	1470.00000000	
AUSTRALIA	10019	Rolltop Desk	480.00000000	
CHINA	10024	Futon Bed	3600.00000000	
CHINA	10023	TV Stand	2640.00000000	
CHINA	10018	Hair Dresser	2408.25000000	
CHINA	10020	Armoire	1785.00000000	
CHINA	10021	Windsor Chair	550.80000000	
CHINA	10019	Rolltop Desk	432.00000000	
CHINA	10022	Nightstand	379.50000000	
CHINA	10001	Cradle Bed	160.00000000	
CHINA	10025	School Chair	157.50000000	
DENMARK	10004	Entertainme...	21825.000000...	
DENMARK	10024	Futon Bed	12870.000000...	
DENMARK	10007	Dinning Table	10933.450000...	
DENMARK	10018	Hair Dresser	9168.25000000	
DENMARK	10006	Settee	6932.00000000	
DENMARK	10021	Windsor Chair	3570.00000000	
DENMARK	10002	Cabinets	3381.00000000	
DENMARK	10016	Washstand	2960.70000000	
DENMARK	10010	Height Bar S...	2562.50000000	
DENMARK	10005	Writer Desk	1757.00000000	
DENMARK	10020	Armoire	1680.00000000	
DENMARK	10014	Futon Chair	1517.00000000	
DENMARK	10011	Dinning Chair	165.00000000	
FRANCE	10018	Hair Dresser	2535.00000000	
GERMANY	10004	Entertainme...	32850.000000...	
GERMANY	10007	Dinning Table	20977.000000...	
GERMANY	10018	Hair Dresser	9802.00000000	
GERMANY	10024	Futon Bed	8909.10000000	
GERMANY	10005	Writer Desk	7063.00000000	
GERMANY	10014	Futon Chair	4331.65000000	
GERMANY	10016	Washstand	4164.15000000	
GERMANY	10021	Windsor Chair	3332.00000000	
GERMANY	10010	Height Bar S...	2275.00000000	
GERMANY	10006	Settee	2224.00000000	
GERMANY	10020	Armoire	1155.00000000	
INDIA	10015	Wardrobe	5625.00000000	
INDIA	10004	Entertainme...	4500.00000000	
INDIA	10007	Dinning Table	2904.00000000	

US	10011	Dinning Chair	3315.00000000
US	10006	Settee	2612.00000000
US	10008	Coffee Table	2090.50000000
US	10001	Cradle Bed	2050.00000000
US	10010	Height Bar S...	1900.00000000
US	10021	Windsor Chair	1778.20000000
US	10022	Nightstand	1702.00000000
US	10016	Washstand	1430.65000000
US	10014	Futon Chair	1357.10000000
US	10009	Stool	1127.00000000
US	10013	Cafe Chair	1037.75000000
US	10025	School Chair	864.75000000

2.2. Write 3 SQL queries, using subqueries (use minimum 3 tables)

2.2.1 Subqueries 1

Business question

PVFC wants to identify which customers have purchased furniture products priced above the company's average unit price. In order to determine which customer segments show greater interest in high-value products.

SQL:

```

SELECT cu.cust_id,
       cu.fname,
       cu.lname,
       c.country_name
FROM gb_cust cu JOIN gb_country c ON c.country_id = cu.country_id
WHERE cu.cust_id IN (
    SELECT DISTINCT o.cust_id
    FROM gb_order o
    JOIN gb_ord_prod op ON o.order_id = op.order_id
    JOIN gb_prod p ON op.prod_id = p.prod_id
    WHERE p.unit_price >
        (SELECT AVG(unit_price) FROM gb_prod)
);

```

Result screenshot:

```
54 * SELECT cu.cust_id,
55      cu.fname,
56      cu.lname,
57      c.country_name
58 FROM gb_cust cu JOIN gb_country c ON c.country_id = cu.country_id
59 WHERE cu.cust_id IN (
60     SELECT DISTINCT o.cust_id
61     FROM gb_order o
62     JOIN gb_ord_prod op ON o.order_id = op.order_id
63     JOIN gb_prod p ON op.prod_id = p.prod_id
64     WHERE p.unit_price >
65           (SELECT AVG(unit_price) FROM gb_prod)
66 )
```

Result Grid

cust_id	fname	lname	country_name
100003	MAYA	Barrington	US
100004	KELLY	Canon	US
100005	JAMIE	Finley	US
100007	HARRY	Hanson	US
100008	Derrick	Snyders	US
100009	Lon	Borther	US
100012	ROBERT	JONES	US
100014	JOSEPH	DAVIS	US
100015	THOMAS	WILSON	US
100016	CHRISTOPHER	MARTIN	US
100020	ANTHONY	CLARK	US
100021	KEVIN	LEWIS	US
100022	GARY	WALKER	US
100024	SCOTT	KING	US
100025	ERIC	WRIGHT	US
100028	BRUCE	ADAMS	US
100029	FRED	CAMPB...	US
100032	Mina	Nylund	UK
100040	ALAN	BAILEY	UK
100045	Calhong	Ding	CHINA
100049	QIANG	Hong	CHINA
100053	KIRAN	RAUT	INDIA
100054	CHARU	SINGH	INDIA
100057	VIPIN	GARG	INDIA
100065	HARSH	KHANNA	INDIA
100070	SALIA	ALLEN	AUSTRALIA
100071	Charles	Fidel	SWITZERLA...
100072	Daniel	Franklin	SWITZERLA...
100073	Jonathan	Greer	SWITZERLA...
100074	Cristina	Garcia	SWITZERLA...
100075	Jason	Henderson	GERMANY
100076	Jasmine	Gentry	GERMANY
100077	Cadlin	Tomes	GERMANY
100078	Michael	Conner	GERMANY
100079	Joshua	Bell	GERMANY
100080	Raymond	Meyer	DENMARK
100081	Juan	Parish	DENMARK
100082	Jeffery	Coleman	DENMARK
100083	Amy	King	DENMARK

Result 17

Action Output 2

	Time	Action	Response	Duration / Fetch Time
2	03:39:28	SELECT cu.cust_id, cu.fname, cu.lname, c.country_name FROM...	39 row(s) returned	0.012 sec / 0.000018...

2.2.2 Subqueries 2

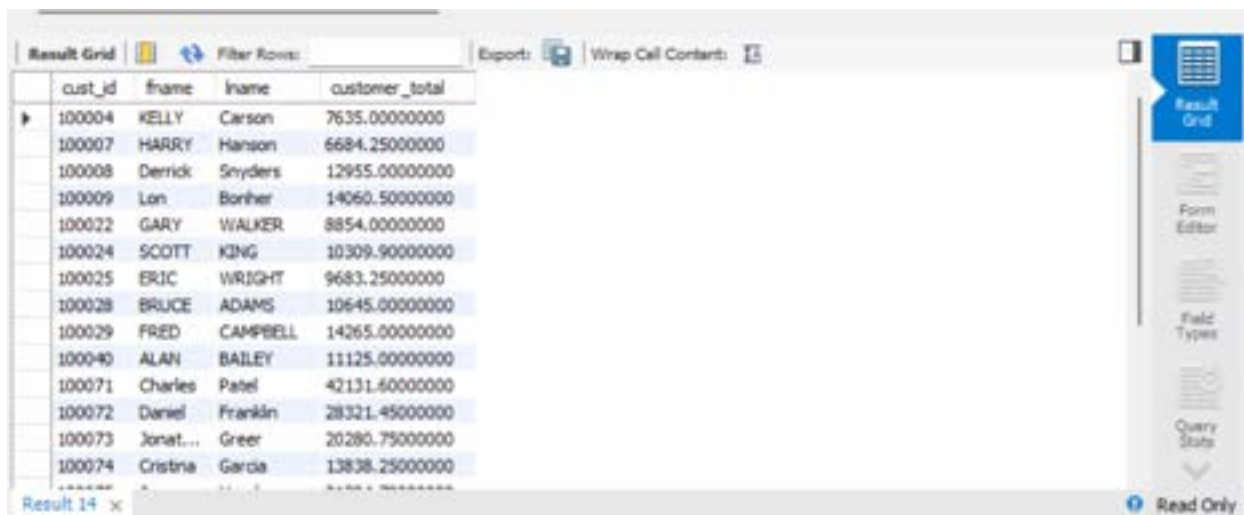
Business question:

Find customers whose total spending is above the average total spending of all customers.

SQL:

```
SELECT
  c.cust_id,
  c.fname,
  c.lname,
  SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) AS customer_total
FROM gb_cust    c
JOIN gb_order    o  ON c.cust_id = o.cust_id
JOIN gb_ord_prod op  ON o.order_id = op.order_id
JOIN gb_prod     p   ON op.prod_id = p.prod_id
GROUP BY
  c.cust_id,
  c.fname,
  c.lname
HAVING
  SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) >
  (
    SELECT AVG(customer_total)
    FROM (
      SELECT
        c2.cust_id,
        SUM(op2.quant * p2.unit_price * (1 - op2.discount_pct/100)) AS
customer_total
      FROM gb_cust    c2
      JOIN gb_order    o2  ON c2.cust_id = o2.cust_id
      JOIN gb_ord_prod op2 ON o2.order_id = op2.order_id
      JOIN gb_prod     p2   ON op2.prod_id = p2.prod_id
      GROUP BY
        c2.cust_id
    ) AS t
  );
```

Result screenshot:



The screenshot shows a database query result grid with the following columns: cust_id, fname, lname, and customer_total. The grid displays 14 rows of data. The interface includes a 'Filter Rows' button, an 'Export' button, and a 'Wrap Cell Contents' button. On the right side, there is a sidebar with options for 'Result Grid', 'Form Editor', 'Field Types', and 'Query Stats'. The bottom status bar indicates 'Result 14' and 'Read Only'.

	cust_id	fname	lname	customer_total
▶	100004	KELLY	Carson	7635.00000000
	100007	HARRY	Hanson	6684.25000000
	100008	Derrick	Snyders	12955.00000000
	100009	Lon	Borher	14060.50000000
	100022	GARY	WALKER	8854.00000000
	100024	SCOTT	KING	10309.90000000
	100025	ERIC	WRIGHT	9683.25000000
	100028	BRUCE	ADAMS	10645.00000000
	100029	FRED	CAMPBELL	14265.00000000
	100040	ALAN	BAILEY	11125.00000000
	100071	Charles	Patel	42131.60000000
	100072	Daniel	Franklin	28321.45000000
	100073	Jonat...	Greer	20280.75000000
	100074	Cristna	Garcia	13838.25000000

2.2.3 Subqueries 3

Business question:

Which customers have individual orders that exceed their own average order value, and what does this tell us about their purchasing behavior?

SQL:

```
SELECT
    o.order_id,
    o.cust_id,
    SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) AS order_total
FROM gb_order o
JOIN gb_ord_prod op ON o.order_id = op.order_id
JOIN gb_prod p ON op.prod_id = p.prod_id
GROUP BY
    o.order_id,
    o.cust_id
HAVING
    SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) >
    (
        SELECT AVG(inner_order_total)
        FROM (
            SELECT
                o2.order_id,
                SUM(op2.quant * p2.unit_price * (1 - op2.discount_pct/100)) AS
            inner_order_total
            FROM gb_order o2
```

```

JOIN gb_ord_prod op2 ON o2.order_id = op2.order_id
JOIN gb_prod p2 ON op2.prod_id = p2.prod_id
WHERE o2.cust_id = o.cust_id
GROUP BY
    o2.order_id
) AS t
);

```

Result screenshot:

order_id	cust_id	order_total
100000001	100001	864.75000000
100000002	100002	789.57750000
100000019	100019	1792.50000000
100000022	100022	8262.50000000
100000003	100003	4483.50000000
100000023	100023	4041.75000000
100000027	100027	293.25000000
100000004	100004	6735.00000000
100000024	100024	9407.50000000
100000028	100028	9600.00000000
100000078	100008	12000.00000000
100000005	100005	4317.00000000
100000025	100025	7688.25000000
100000029	100029	7425.00000000

2.3. Write 2 SQL queries using analytical functions , one using ROLLUP and other using RANK() functions.

2.3.1 ROLLUP

Business question:

For each customer, how many items have they ordered in total, and what is the overall total quantity ordered by all customers?

SQL:

```

SELECT
    c.cust_id,
    CONCAT(c.fname, ' ', c.lname) AS customer_name,
    SUM(op.quant) AS total_quantity
FROM gb_cust c
JOIN gb_order o ON c.cust_id = o.cust_id
JOIN gb_ord_prod op ON o.order_id = op.order_id
GROUP BY c.cust_id, c.fname, c.lname WITH ROLLUP;

```

Result screenshot:

The screenshot displays a database management interface. On the left, a 'SCHEMAS' pane shows a tree view of database objects, including tables like 'gb_cust', 'gb_order', and 'gb_ord_prod'. The main area shows a SQL query being executed. The query is as follows:

```
SELECT
  c.cust_id,
  CONCAT(c.fname, ' ', c.lname) AS customer_name,
  SUM(op.quant) AS total_quantity
FROM
  gb_cust c
JOIN
  gb_order o ON c.cust_id = o.cust_id
JOIN
  gb_ord_prod op ON o.order_id = op.order_id
GROUP BY c.cust_id, c.fname, c.lname WITH ROLLUP;
```

Below the query, the 'Result Grid' shows the output of the query. The grid has four columns: 'cust_id', 'customer_name', and 'total_quantity'. The results are as follows:

cust_id	customer_name	total_quantity
100001	MIKE Ashton	13
100001	MIKE	13
100001	MIKE	13
100002	LUCY Baron	5
100002	MIKE	5
100002	MIKE	5
100003	MAYA Barrington	9
100003	MIKE	9
100003	MIKE	9
100004	KELLY Carson	7
100004	MIKE	7
100004	MIKE	7

Result Grid				Filter Rows	Export	Wrap Cell Contents
	cust_id	customer_name	total_quantity			
▶	100001	MIKE Ashton	13			
	100001	NULL	13			
	100001	NULL	13			
	100002	LUCY Baron	5			
	100002	NULL	5			
	100002	NULL	5			
	100003	MAYA Barrington	9			
	100003	NULL	9			
	100003	NULL	9			
	100004	KELLY Carson	7			
	100004	NULL	7			
	100004	NULL	7			
	100005	JAMIE Finley	8			
	100005	NULL	8			
	100005	NULL	8			
	100006	NICK Finley	10			
	100006	NULL	10			
	100006	NULL	10			
	100007	HARRY Hanson	13			
	100007	NULL	13			
	100007	NULL	13			
	100008	Derrick Snyders	6			
	100008	NULL	6			
	100008	NULL	6			
	100009	Lon Bonher	8			
	100009	NULL	8			
	100009	NULL	8			
	100010	JAMES SMITH	7			
	100010	NULL	7			
	100010	NULL	7			

	cust_id	customer_name	total_quantity
	100010	JAMES SMITH	7
	100010	NULL	7
	100010	NULL	7
	100011	Flannery Newton	8
	100011	NULL	8
	100011	NULL	8
	100012	ROBERT JONES	15
	100012	NULL	15
	100012	NULL	15
	100013	RICHARD MILLER	14
	100013	NULL	14
	100013	NULL	14
	100014	JOSEPH DAVIS	11
	100014	NULL	11
	100014	NULL	11
	100015	THOMAS WILSON	12
	100015	NULL	12
	100015	NULL	12
	100016	CHRISTOPHER ...	12
	100016	NULL	12
	100016	NULL	12
	100017	MARK THOMPSON	5
	100017	NULL	5
	100017	NULL	5
	100018	DONALD TRUMP	6
	100018	NULL	6
	100018	NULL	6
	100019	BRIAN HARRIS	6
	100019	NULL	6
	100019	NULL	6

	cust_id	customer_name	total_quantity
	100019	NULL	6
	100020	ANTHONY CLARK	5
	100020	NULL	5
	100020	NULL	5
	100021	KEVIN LEWIS	8
	100021	NULL	8
	100021	KEVIN LEWIS	8
	100022	GARY WALKER	13
	100022	NULL	13
	100022	NULL	13
	100023	JOSE SANCHEZ	10
	100023	NULL	10
	100023	NULL	10
	100024	SCOTT KING	14
	100024	NULL	14
	100024	NULL	14
	100025	ERIC WRIGHT	12
	100025	NULL	12
	100025	NULL	12
	100026	WILLIE HILL	3
	100026	NULL	3
	100026	NULL	3
	100027	NICHOLAS FLO...	2
	100027	NULL	2
	100027	NULL	2
	100028	BRUCE ADAMS	6
	100028	NULL	6
	100028	NULL	6
	100029	FRED CAMPBELL	8
	100029	NULL	8

	cust_id	customer_name	total_quantity
	100029	FRED CAMPBELL	8
	100029	NULL	8
	100029	NULL	8
	100030	HOWARD CARTER	6
	100030	NULL	6
	100030	NULL	6
	100031	STEVE HALL	5
	100031	NULL	5
	100031	NULL	5
	100032	Miina Nylund	3
	100032	NULL	3
	100032	NULL	3
	100033	Kauri Anaru	1
	100033	NULL	1
	100033	NULL	1
	100034	Cansu Peynirci	5
	100034	NULL	5
	100034	NULL	5
	100035	RUSSELL RYDER	1
	100035	NULL	1
	100035	NULL	1
	100036	BOBBY PARKER	4
	100036	NULL	4
	100036	NULL	4
	100037	VICTOR TURNER	2
	100037	NULL	2
	100037	NULL	2
	100038	ERNEST COOK	1
	100038	NULL	1
	100038	NULL	1

	cust_id	customer_name	total_quantity
	100038	NULL	1
	100038	NULL	1
	100039	PHILLIP COOPER	3
	100039	NULL	3
	100039	NULL	3
	100040	ALAN BAILEY	5
	100040	NULL	5
	100040	NULL	5
	100041	SHAWN REED	7
	100041	NULL	7
	100041	NULL	7
	100042	SEAN COX	4
	100042	NULL	4
	100042	NULL	4
	100043	Ai Chen	3
	100043	NULL	3
	100043	NULL	3
	100044	Bao Cheng	1
	100044	NULL	1
	100044	NULL	1
	100045	Caihong Ding	1
	100045	NULL	1
	100045	NULL	1
	100046	Chun Deng	2
	100046	NULL	2
	100046	NULL	2
	100047	Hua Fang	3
	100047	NULL	3
	100047	NULL	3
	100048	Juan Guo	6

	cust_id	customer_name	total_quantity
	100048	NULL	6
	100048	NULL	6
	100049	QIANG Hong	5
	100049	NULL	5
	100049	NULL	5
	100050	WEI Huang	6
	100050	NULL	6
	100050	NULL	6
	100051	JIE Jiang	1
	100051	NULL	1
	100051	NULL	1
	100052	KARAN MATHUR	1
	100052	NULL	1
	100052	NULL	1
	100053	KIRAN RAUT	1
	100053	NULL	1
	100053	NULL	1
	100054	CHARU SINGH	1
	100054	NULL	1
	100054	NULL	1
	100055	KUSHAL RAO	2
	100055	NULL	2
	100055	NULL	2
	100056	ARVI PATEL	1
	100056	NULL	1
	100056	NULL	1
	100057	VIPIN GARG	3
	100057	NULL	3
	100057	NULL	3
	100058	SIMRAN KAUR	5

	cust_id	customer_name	total_quantity
	100057	NULL	3
	100058	SIMRAN KAUR	5
	100058	NULL	5
	100058	NULL	5
	100059	VISHAL RAWAT	7
	100059	NULL	7
	100059	NULL	7
	100060	DISHA KHURANA	4
	100060	NULL	4
	100060	NULL	4
	100061	Chandrakant Ch...	4
	100061	NULL	4
	100061	NULL	4
	100062	AMRIT RAO	1
	100062	NULL	1
	100062	NULL	1
	100063	TARAL JANA	1
	100063	NULL	1
	100063	NULL	1
	100064	KAJAL SINGH	4
	100064	NULL	4
	100064	NULL	4
	100065	HARSH KHANNA	3
	100065	NULL	3
	100065	NULL	3
	100066	Gavino Bove	2
	100066	NULL	2
	100066	NULL	2
	100067	Angela Jephson	1
	100067	NULL	1

	cust_id	customer_name	total_quantity
	100069	NULL	1
	100070	TALIA ALLEN	1
	100070	NULL	1
	100070	NULL	1
	100071	Charles Patel	35
	100071	NULL	35
	100071	NULL	35
	100072	Daniel Franklin	35
	100072	NULL	35
	100072	NULL	35
	100073	Jonathan Greer	33
	100073	NULL	33
	100073	NULL	33
	100074	Cristina Garcia	24
	100074	NULL	24
	100074	NULL	24
	100075	Jason Henderson	25
	100075	NULL	25
	100075	NULL	25
	100076	Jasmine Gentry	29
	100076	NULL	29
	100076	NULL	29
	100077	Caitlin Torres	29
	100077	NULL	29
	100077	NULL	29
	100078	Michael Conner	30
	100078	NULL	30
	100078	NULL	30
	100079	Joshua Bell	33
	100079	NULL	33

100078	HULL	30
100079	Joshua Bell	33
100079	HULL	33
100079	HULL	33
100080	Raymond Meyer	31
100080	HULL	31
100080	HULL	31
100081	Juan Parrish	46
100081	HULL	46
100081	HULL	46
100082	Jeffery Coleman	29
100082	HULL	29
100082	HULL	29
100083	Amy King	31
100083	HULL	31
100083	HULL	31
HULL	HULL	787

2.3.2 RANK

Business question:

Which products are ordered the most, and what is the ranking of each product based on total quantity ordered?

SQL:

```
SELECT b.prod_id, b.name AS prod_name, p.total_quantity,
       RANK() OVER (ORDER BY p.total_quantity DESC) AS quantity_rank
FROM   gb_prod b,( SELECT prod_id, SUM(quant) AS total_quantity
                   FROM   gb_ord_prod
                   GROUP BY prod_id) p
WHERE  b.prod_id = p.prod_id
ORDER BY quantity_rank;
```

Result screenshot:

```

335 • SELECT b.prod_id, b.name AS prod_name, p.total_quantity,
336         RANK() OVER (ORDER BY p.total_quantity DESC) AS quantity_rank
337 FROM   gb_prod b,( SELECT prod_id, SUM(quant) AS total_quantity
338                   FROM   gb_ord_prod
339                   GROUP BY prod_id) p
340 WHERE  b.prod_id = p.prod_id
341 ORDER BY quantity_rank;
342
343
344

```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 

	prod_id	prod_name	total_quantity	quantity_rank
▶	10007	Dinning Table	78	1
	10018	Hair Dresser	61	2
	10014	Futon Chair	58	3
	10005	Writer Desk	57	4
	10006	Settee	56	5
	10016	Washstand	48	6
	10010	Height Bar Stool	44	7
	10024	Futon Bed	44	7
	10021	Windsor Chair	41	9
	10004	Entertainment Center	36	10
	10011	Dinning Chair	35	11

Result Grid  Filter Rows: Export:  Wrap Cell Content: 

	prod_id	prod_name	total_quantity	quantity_rank
	10004	Entertainment Center	36	10
	10011	Dinning Chair	35	11
	10009	Stool	28	12
	10002	Cabinets	24	13
	10025	School Chair	21	14
	10020	Armoire	19	15
	10023	TV Stand	19	15
	10017	Bookcase	16	17
	10015	Wardrobe	15	18
	10008	Coffee Table	14	19
	10012	Accent Chair	14	19
	10022	Nightstand	14	19
	10001	Cradle Bed	12	22
	10003	Couch	12	22
	10013	Cafe Chair	11	24
	10019	Rolltop Desk	10	25

3. Content of Project Phase 3

3.1 Two non-unique index

3.1.1 cust_id on table gb_order

CREATE INDEX idx_gb_order_cust_id
ON gb_order (cust_id);

The screenshot displays the MySQL Workbench interface. The top pane shows the SQL editor with the following code:

```
1 use gb_pvf;
2 CREATE INDEX idx_gb_order_cust_id
3 ON gb_order (cust_id);
```

The bottom pane shows the 'Output' tab with the following log:

#	Time	Action	Message
257	20:11:42	CREATE INDEX idx_gb_order_cust_id ON gb_order (cust_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings:
258	20:12:09	SELECT * FROM gb_pvf.gb_order LIMIT 0, 50000	83 row(s) returned

Below the output, the 'Table Name' is set to 'gb_order' and the 'Schema' is 'gb_pvf'. The 'Index Name' is 'idx_gb_order_cust_id' and the 'Type' is 'INDEX'. The 'Index Columns' table is empty. The 'Index Options' section shows 'Storage Type' as 'InnoDB', 'Key Block Size' as '1', 'Parser' as '1', and 'Visible' as '1'. The 'Index Comment' field is empty.

3.1.2 Iname on table gb_cust

CREATE INDEX idx_gb_cust_lname
ON gb_cust (lname);

```
5 * CREATE INDEX idx_gb_cust_lname
6 ON gb_cust (lname);
7
```

Output

Action Output

270 20:27:05 CREATE INDEX idx_gb_cust_lname ON gb_cust (lname) 0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Table Name: gb_cust Schema: gb_privc

Charset/Collation: utf8mb4 utf8mb4_0900_ai_ci Engine: InnoDB

Comments:

Index Name	Type
PRIMARY	PRIMARY
gb_cust_gb_country_fk	INDEX
idx_gb_cust_lname	INDEX

Index Columns			
Column	#	Order	Length

Index Options

Storage Type:

Key Block Size:

Parser:

Visible: ☐

Index Comment:

Use the right mouse button on the index grid to bring up the popup menu

Columns Indexes Foreign Keys Triggers Partitioning Options

3.2 Two composite Indexes

3.2.1 Composite Index on gb_ord_prod(order_id, prod_id)

CREATE INDEX idx_gb_ord_prod_order_prod

ON gb_ord_prod (order_id, prod_id);

The screenshot shows a SQL IDE interface. The top pane contains the following SQL code:

```
10 • CREATE INDEX idx_gb_ord_prod_order_prod
11 ON gb_ord_prod (order_id, prod_id);
12
```

The bottom pane shows the execution output:

Time	Action	Response	Duration / F
21:43:32	CREATE INDEX idx_gb_cust_country_state ON gb_cust (country_id, state)	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.023 sec

3.2.2 Composite Index on gb_cust(country_id, state)

CREATE INDEX idx_gb_cust_country_state

ON gb_cust (country_id, state);

The screenshot shows a SQL IDE interface. The top pane contains the following SQL code:

```
13 • CREATE INDEX idx_gb_cust_country_state
14 ON gb_cust (country_id, state);
15
16 • ALTER TABLE gb_order
```

The bottom pane shows the execution output:

Time	Action	Response	Duration / F
21:43:32	CREATE INDEX idx_gb_cust_country_state ON gb_cust (country_id, state)	0 row(s) affected Records: 0 Duplicates: 0 Warnings...	0.023 sec

```

23 * SELECT
24     table_schema,
25     table_name,
26     index_name,
27     non_unique,
28     seq_in_index,
29     column_name,
30     sub_part,
31     index_type
32 FROM information_schema.statistics
33 WHERE table_schema = 'gb_order'
34 AND table_name LIKE 'gb_order%'
35 ORDER BY
36     table_name,
37     index_name,
38     seq_in_index

```

TABLE_SCHEMA	TABLE_NAME	INDEX_NAME	NON_UNIQUE	SEQ_IN_INDEX	COLUMN_NAME	SUB_PART	INDEX_TYPE
gb_order	gb_order	PRIMARY	0	1	country_id		BTREE
gb_order	gb_order	idx_gb_order_country_state	1	1	country_id		BTREE
gb_order	gb_order	idx_gb_order_country_state	1	2	state		BTREE
gb_order	gb_order	idx_gb_order_name	1	1	name		BTREE
gb_order	gb_order	idx_name_upper	1	1	name		BTREE
gb_order	gb_order	PRIMARY	0	1	cust_id		BTREE
gb_order	gb_order	PRIMARY	0	1	order_id		BTREE
gb_order	gb_order	idx_order_gb_order_id	1	1	order_id		BTREE
gb_order	gb_order	idx_order_gb_order_id	1	1	order_id		BTREE
gb_order	gb_order	idx_order_gb_order_id	1	1	order_id		BTREE
gb_order	gb_order	idx_order_gb_order_id	1	1	order_id		BTREE
gb_order	gb_order	PRIMARY	0	1	order_id		BTREE
gb_order	gb_order	PRIMARY	0	2	order_id		BTREE
gb_order	gb_order	PRIMARY	0	3	order_id		BTREE
gb_order	gb_order	idx_order_order_id	1	1	order_id		BTREE
gb_order	gb_order	idx_order_order_id	1	1	order_id		BTREE
gb_order	gb_order	PRIMARY	0	1	order_id		BTREE
gb_order	gb_order	PRIMARY	0	1	order_id		BTREE

3.3 Two function based index

3.3.1 Index idx_order_month

ALTER TABLE gb_order

ADD INDEX idx_order_month ((MONTH(order_date)));

SHOW INDEXES FROM gb_order;

```

16 * ALTER TABLE gb_order
17     ADD INDEX idx_order_month ((MONTH(order_date)));
18
19 * SHOW INDEXES FROM gb_order;

```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible
gb_order	0	PRIMARY	1	order_id	A	153				BTREE			YES
gb_order	1	gb_order_gb_order_id	1	order_id	A	83				BTREE			YES
gb_order	1	idx_order_month	1	order_id	A	11			YES	BTREE			YES

Time	Action	Response	Duration / Fetch Time
01:14:21	ALTER TABLE gb_order ADD INDEX idx_order_month ((MONTH(order_id)));	0 row(s) affected Records: 0 Duplicates: 0 Warnin...	0.047 sec
01:14:21	SHOW INDEXES FROM gb_order	3 row(s) returned	0.0099 sec / 0.0000...

3.3.2 Index idx_lname_upper

```
ALTER TABLE gb_cust
```

```
ADD INDEX idx_lname_upper ((UPPER(lname)));
```

```
SHOW INDEXES FROM gb_cust;
```

```
21 * ALTER TABLE gb_cust
22   ADD INDEX idx_lname_upper ((UPPER(lname)));
23
24 * SHOW INDEXES FROM gb_cust;
25
```

The screenshot shows a database client interface with a 'Result Grid' and an 'Action Output' pane. The 'Result Grid' displays the output of the 'SHOW INDEXES FROM gb_cust;' command, showing three indexes: a primary key on cust_id, a foreign key on country_id, and a new index on the uppercased lname. The 'Action Output' pane shows the execution details for both commands.

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible
gb_cust	0	PRIMARY	1	cust_id	A	83				STREE			YES
gb_cust	1	gb_cust_gb_country_fk	1	country_id	A	10				STREE			YES
gb_cust	1	idx_lname_upper	1	lname	A	77			YES	STREE			YES

	Time	Action	Response	Duration / Fetch Time
4	01:15:08	ALTER TABLE gb_cust ADD INDEX idx_lname_upper ((UPPER(lname)))	0 row(s) affected Records: 0 Duplicates: 0 Warnin...	0.068 sec
5	01:15:08	SHOW INDEXES FROM gb_cust	3 row(s) returned	0.0016 sec / 0.00001...

3.4 List of All Indexes Using SQL

3.4.1 SQL

```
/*
```

List all indexes for PVFC tables of group GB

WHW:

- What: List index details for all GB_% tables in schema GB_PVFC on this instance.
- How: Read index metadata from INFORMATION_SCHEMA.STATISTICS with TABLE_SCHEMA = 'gb_pvfc'.
- Why: Verify that all 6 new indexes exist together with existing PK/UK/FK indexes.

```
*/
```

```
SELECT
```

```
  table_schema,  
  table_name,  
  index_name,  
  non_unique,  
  seq_in_index,  
  column_name,  
  sub_part,  
  index_type
```

```
FROM information_schema.statistics
```

```
WHERE table_schema = 'gb_pvfc'
```

```

AND table_name LIKE 'gb_%'
ORDER BY
    table_name,
    index_name,
    seq_in_index;

```

3.4.2 Screenshot

```

23 * SELECT
24     table_schema,
25     table_name,
26     index_name,
27     non_unique,
28     seq_in_index,
29     column_name,
30     sub_part,
31     index_type
32 FROM information_schema.statistics
33 WHERE table_schema = 'gb_prod'
34 AND table_name LIKE 'gb_%'
35 ORDER BY
36     table_name,
37     index_name,
38     seq_in_index;

```

TABLE_SCHEMA	TABLE_NAME	INDEX_NAME	NON_UNIQUE	SEQ_IN_INDEX	COLUMN_NAME	SUB_PART	INDEX_TYPE
gb_prod	gb_country	PRIMARY	0	1	country_id	0000	BTREE
gb_prod	gb_cust	idx_gb_cust_country_state	1	1	country_id	0000	BTREE
gb_prod	gb_cust	idx_gb_cust_country_state	1	2	state	0000	BTREE
gb_prod	gb_cust	idx_gb_cust_name	1	1	name	0000	BTREE
gb_prod	gb_cust	idx_name_upper	1	1	name	0000	BTREE
gb_prod	gb_cust	PRIMARY	0	1	cust_id	0000	BTREE
gb_prod	gb_branch	PRIMARY	0	1	branch_id	0000	BTREE
gb_prod	gb_order_prod	gb_order_prod_gb_branch_id	1	1	branch_id	0000	BTREE
gb_prod	gb_order_prod	gb_order_prod_gb_prod_id	1	1	prod_id	0000	BTREE
gb_prod	gb_order_prod	idx_gb_order_prod_order_prod	1	1	order_id	0000	BTREE
gb_prod	gb_order_prod	idx_gb_order_prod_order_prod	1	2	prod_id	0000	BTREE
gb_prod	gb_order_prod	PRIMARY	0	1	order_id	0000	BTREE
gb_prod	gb_order_prod	PRIMARY	0	2	prod_id	0000	BTREE
gb_prod	gb_order	idx_gb_order_order_id	1	1	order_id	0000	BTREE
gb_prod	gb_order	idx_order_product	1	1	prod_id	0000	BTREE
gb_prod	gb_order	PRIMARY	0	1	order_id	0000	BTREE
gb_prod	gb_prod	PRIMARY	0	1	prod_id	0000	BTREE

statistics 1

Run Output 1

Time	Action	Response
0	21:55:01 SELECT table_schema, table_name, index_name, non_unique, seq_in_index, column_name, sub_part, index_type FROM information_schema.statistics WHERE table_...	16 rows(s) returned

3.5 Screenshot of indexes from MySQL workbench

Table	Name	Unique	Index	Index Comment	Columns	Seq in Index	Packed	Cardi...	Cardin...	Sub-part	ISLS	Comment
gh_country	PRIMARY	Yes	BTREE		country_id	1		A	55			
gh_cust	PRIMARY	Yes	BTREE		cust_id	1		A	83			
gh_cust		No	BTREE		name	1		A	77			
gh_cust		No	BTREE		country_id	1		A	55			
gh_cust		No	BTREE		state	2		A	40			
gh_cust		No	BTREE		name_upper	1		A	77			YES
gh_fresh	PRIMARY	Yes	BTREE		fresh_id	1		A	20			
gh_int_prod	PRIMARY	Yes	BTREE		order_id	1		A	151			
gh_int_prod	PRIMARY	Yes	BTREE		prod_id	2		A	255			
gh_int_prod	PRIMARY	Yes	BTREE		fresh_id	3		A	205			
gh_int_prod		No	BTREE		fresh_id	1		A	25			
gh_int_prod		No	BTREE		prod_id	1		A	25			
gh_int_prod		No	BTREE		order_id	1		A	151			
gh_int_prod		No	BTREE		prod_id	2		A	255			
gh_int_prod		No	BTREE		order_id	1		A	151			
gh_order	PRIMARY	Yes	BTREE		order_id	1		A	151			
gh_order		No	BTREE		cust_id	1		A	83			
gh_order		No	BTREE		order_date	1		A	11			YES
gh_prod	PRIMARY	Yes	BTREE		prod_id	1		A	25			

4. Database procedure to collect statistics of all tables, and call the procedure.

4.1 Procedure code and Call statement

SQL COMMAND

DELIMITER \$\$

CREATE PROCEDURE script_to_analyse_all_tables()

BEGIN

-- declare variables

DECLARE finished INTEGER DEFAULT 0;

DECLARE this_table_name VARCHAR(256);

DECLARE full_statement VARCHAR(10000);

-- declare cursor for table names

DECLARE cursor_table_name CURSOR FOR

SELECT table_name FROM INFORMATION_SCHEMA.TABLES WHERE
TABLE_SCHEMA = '[gb_pvfc]';

-- declare NOT FOUND handler

DECLARE CONTINUE HANDLER FOR NOT FOUND SET finished = 1;

-- open the cursor

OPEN cursor_table_name;

myLoop: LOOP

FETCH cursor_table_name INTO this_table_name;

```

IF finished = 1 THEN
LEAVE myLoop;
END IF;
SET full_statement = concat( IFNULL(full_statement,"), "ANALYZE TABLE ",
this_table_name, ";");
END LOOP myLoop;
-- close the cursor
CLOSE cursor_table_name;
-- view results:
SELECT full_statement as "run the following statement:";

END $$
DELIMITER ;

-- Call the procedure to run
CALL script_to_analyse_all_tables;

```

SCREEN SHOT

```

5 DELIMITER $$
6
7 * CREATE PROCEDURE script_to_analyse_all_tables()
8 BEGIN
9
10     DECLARE finished INTEGER DEFAULT 0;
11     DECLARE this_table_name VARCHAR(250);
12     DECLARE full_statement VARCHAR(10000);
13
14     DECLARE cursor_table_name CURSOR FOR
15         SELECT table_name
16         FROM INFORMATION_SCHEMA.TABLES
17         WHERE TABLE_SCHEMA = 'gb_pref';
18
19
20     DECLARE CONTINUE HANDLER FOR NOT FOUND SET finished = 1;
21
22
23     OPEN cursor_table_name;
24
25     myLoop: LOOP
26         FETCH cursor_table_name INTO this_table_name;
27
28         IF finished = 1 THEN
29             LEAVE myLoop;
30         END IF;
31
32         SET full_statement = concat(IFNULL(full_statement, ''), "ANALYZE TABLE ", this_table_name, ";");
33
34     END LOOP myLoop;
35
36     CLOSE cursor_table_name;
37
38     SELECT full_statement as "run the following statement:";
39
40 END $$
41 DELIMITER ;
42
43 * -- Call the procedure to run
44 CALL script_to_analyse_all_tables;
45

```

Result Grid

Filter Rows: Search

run the following statement:

ANALYZE TABLE gb_country;ANALYZE TABLE gb_country_s...

Result 12

Action Output

	Time	Action	Response
87	00:49:47	-- Call the procedure to run CALL script_to_analyse_all_tables	1 row(s) returned
88	00:50:10	CALL script_to_analyse_all_tables	1 row(s) returned

4.2 Result of following dictionary query for index statistics

```

50 * SELECT TABLE_SCHEMA,
51      TABLE_NAME, INDEX_NAME, SEQ_IN_INDEX, COLUMN_NAME, CARDINALITY,
52      INDEX_TYPE
53 FROM INFORMATION_SCHEMA.STATISTICS
54 WHERE TABLE_SCHEMA = 'gb_pvtc';

```

TABLE_SCHEMA	TABLE_NAME	INDEX_NAME	SEQ_IN_INDEX	COLUMN_NAME	CARDINALITY	INDEX_TYPE
gb_pvtc	gb_country	PRIMARY	1	country_id	10	BTREE
gb_pvtc	gb_fresh	PRIMARY	1	fresh_id	20	BTREE
gb_pvtc	gb_prod	PRIMARY	1	prod_id	25	BTREE
gb_pvtc	gb_order	PRIMARY	1	order_id	153	BTREE
gb_pvtc	gb_order	idx_gb_order_cust_id	1	cust_id	83	BTREE
gb_pvtc	gb_order	idx_order_month	1	month	11	BTREE
gb_pvtc	gb_ord_prod	PRIMARY	1	order_id	151	BTREE
gb_pvtc	gb_ord_prod	PRIMARY	2	prod_id	295	BTREE
gb_pvtc	gb_ord_prod	PRIMARY	3	fresh_id	300	BTREE
gb_pvtc	gb_ord_prod	idx_ord_prod_gb_fresh_fk	1	fresh_id	20	BTREE
gb_pvtc	gb_ord_prod	idx_ord_prod_gb_prod_fk	1	prod_id	25	BTREE
gb_pvtc	gb_ord_prod	idx_gb_ord_prod_order_id	1	order_id	151	BTREE
gb_pvtc	gb_ord_prod	idx_gb_ord_prod_order_id	2	prod_id	295	BTREE
gb_pvtc	gb_cust	PRIMARY	1	cust_id	83	BTREE
gb_pvtc	gb_cust	idx_gb_cust_fname	1	fname	77	BTREE
gb_pvtc	gb_cust	idx_gb_cust_country_state	2	state	40	BTREE
gb_pvtc	gb_cust	idx_gb_cust_country_state	1	country_id	10	BTREE
gb_pvtc	gb_cust	idx_fname_upper	1	fname	77	BTREE

STATISTICS 13

Action Output 2

	Time	Action	Response
88	00:50:10	CALL script_to_analyze_all_tables	1 row(s) returned
89	00:53:55	SELECT TABLE_SCHEMA, TABLE_NAME, INDEX_NAME, SEQ_IN_INDEX, COLUMN_NAME, CARDINALITY, INDEX_TYPE FROM INFORMATION_SCHEMA.STATISTICS WHERE TABLE_SCHEMA = 'gb_pvtc';	18 row(s) returned

5. Screenshots of each query, and corresponding execution plan

5.1 SQL QUERY 1 (Table Join 1)

Business question:

For each PVFC product, which countries demonstrate genuine demand (sales exceeding 10 units) sufficient to justify

SQL:

```

SELECT p.prod_id,
p.name AS product_name,
c.country_name,
SUM(op.quant) AS total_quantity
FROM gb_prod p
JOIN gb_ord_prod op ON p.prod_id = op.prod_id
JOIN gb_order o ON op.order_id = o.order_id
JOIN gb_cust cu ON o.cust_id = cu.cust_id
JOIN gb_country c ON cu.country_id = c.country_id
GROUP BY p.prod_id, p.name, c.country_name
HAVING SUM(op.quant) > 10

```

ORDER BY total_quantity DESC, c.country_name;

SCREENSHOT

```

56 -- Fp2-a1
57 • SELECT p.prod_id,
58       p.name AS product_name,
59       c.country_name,
60       SUM(op.quant) AS total_quantity
61 FROM   gb_prod p
62 JOIN   gb_ord_prod op ON p.prod_id = op.prod_id
63 JOIN   gb_order o ON op.order_id = o.order_id
64 JOIN   gb_cust cu ON o.cust_id = cu.cust_id
65 JOIN   gb_country c ON cu.country_id = c.country_id
66 GROUP BY p.prod_id, p.name, c.country_name
67 HAVING SUM(op.quant) > 10
68 ORDER BY total_quantity DESC, c.country_name;

```

#	Field	Schema	Table	Type	Character Set	Display Size	Precision	Scale
1	prod_id	gb_pvfc	gb_prod	INT	binary	11	5	0
2	product_name	gb_pvfc	gb_prod	VARCHAR	utf8mb4	100	20	0
3	country_name	gb_pvfc	gb_country	VARCHAR	utf8mb4	50	11	0
4	total_quantity			DECIMAL	binary	28	2	0

STATISTICS 13 Result 14

Action Output

	Time	Action	Response
90	01:00:34	EXPLAIN SELECT p.prod_id, p.name AS product_name, c.country_name, SUM(op.quant) AS total_quantity FROM gb_prod p JOIN...	OK
91	01:00:37	SELECT p.prod_id, p.name AS product_name, c.country_name, SUM(op.quant) AS total_quantity FROM gb_prod p JOIN gb_ord_p...	33 row(s) returned

Tabular Explain

id	select_type	table	partitions	type	possible_keys	key	key...	ref
1	SIMPLE	c		ALL	PRIMARY			
1	SIMPLE	cu		ref	PRIMARY,gb_cust_gb_country_fk	gb_cust_gb_country_fk	1	gb_pvfc.c.country_id
1	SIMPLE	o		ref	PRIMARY,idx_gb_order_cust_id	idx_gb_order_cust_id	4	gb_pvfc.cu.cust_id
1	SIMPLE	op		ref	PRIMARY,gb_ord_prod_gb_prod_fk	PRIMARY	8	gb_pvfc.o.order_id
1	SIMPLE	p		eq_ref	PRIMARY		4	gb_pvfc.op.prod_id

key...	ref	rows	filtered	Extra
		10	100.00	Using temporary; Using filesort
1	gb_pvfc.c.country_id	8	100.00	Using index
4	gb_pvfc.cu.cust_id	1	100.00	Using index
8	gb_pvfc.o.order_id	1	100.00	
4	gb_pvfc.op.prod_id	1	100.00	

5.2 SQL QUERY 2 (Table Join 2)

Business question: Analyze detailed sales by geography and product attributes

SQL:

```
SELECT
co.country_name,
c.cust_id,
c.fname,
c.lname,
o.order_id,
o.order_date,
p.prod_id,
p.name AS prod_name,
f.finish_id,
f.shade AS finish_shade,
op.quant,
p.unit_price,
op.discount_pct,
(op.quant * p.unit_price * (1 - op.discount_pct/100)) AS line_amount
FROM gb_country
co
JOIN gb_cust c ON co.country_id = c.country_id
JOIN gb_order o ON c.cust_id = o.cust_id
JOIN gb_ord_prod op ON o.order_id = op.order_id
JOIN gb_prod p ON op.prod_id = p.prod_id
JOIN gb_finish f ON op.finish_id = f.finish_id
ORDER BY
co.country_name,
c.cust_id,
o.order_id,
p.prod_id;
```

SCREENSHOT

```

71 * SELECT
72     co.country_name,
73     c.cust_id,
74     c.fname,
75     c.lname,
76     o.order_id,
77     o.order_date,
78     p.prod_id,
79     p.name AS prod_name,
80     f.finish_id,
81     f.shade AS finish_shade,
82     op.quant,
83     p.unit_price,
84     op.discount_pct,
85     (op.quant * p.unit_price * (1 - op.discount_pct/100)) AS line_amount
86 FROM gb_country co
87 JOIN gb_cust c ON co.country_id = c.country_id
88 JOIN gb_order o ON c.cust_id = o.cust_id
89 JOIN gb_ord_prod op ON o.order_id = op.order_id
90 JOIN gb_prod p ON op.prod_id = p.prod_id
91 JOIN gb_finish f ON op.finish_id = f.finish_id
92 ORDER BY
93     co.country_name,
94     c.cust_id,
95     o.order_id,
96     p.prod_id,
97

```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	co		ALL	PRIMARY				10	100.00	Using temporary; Using filesort
1	SIMPLE	c		ref	PRIMARY,idx_gb_cust_country...	idx_gb_cust_country_state	1	gb_pvfc.co.country_id	8	100.00	
1	SIMPLE	o		ref	PRIMARY,idx_gb_order_cust_id	idx_gb_order_cust_id	4	gb_pvfc.c.cust_id	1	100.00	
1	SIMPLE	op		ref	PRIMARY,gb_ord_prod_gb_fin...	PRIMARY	8	gb_pvfc.o.order_id	1	100.00	
1	SIMPLE	f		eq_ref	PRIMARY	PRIMARY	2	gb_pvfc.op.finish_id	1	100.00	
1	SIMPLE	p		eq_ref	PRIMARY	PRIMARY	4	gb_pvfc.op.prod_id	1	100.00	

STATISTICS 13 Result 14 Explain

Action Output 2

Time Action Response

92 01:09:01 EXPLAIN SELECT co.country_name, c.cust_id, c.fname, c.lname, o.order_id, o.order_date, p.prod_id, p.name AS prod_name, f.finish_id, f.shade AS finish_shade, op.quant, p.unit_price, op.discount_pct, (...) OK

Tabular Explain *

id	select_type	table	partitions	type	possible_keys	key	key...
1	SIMPLE	co		ALL	PRIMARY		
1	SIMPLE	c		ref	PRIMARY,gb_cust_gb_country_fk	gb_cust_gb_country_fk	1
1	SIMPLE	o		ref	PRIMARY,idx_gb_order_cust_id	idx_gb_order_cust_id	4
1	SIMPLE	op		ref	PRIMARY,gb_ord_prod_gb_fin...	PRIMARY	8
1	SIMPLE	f		eq_ref	PRIMARY	PRIMARY	2
1	SIMPLE	p		eq_ref	PRIMARY	PRIMARY	4

key...	ref	rows	filtered	Extra
		10	100.00	Using temporary; Using filesort
1	gb_pvfc.co.country_id	8	100.00	
4	gb_pvfc.c.cust_id	1	100.00	
8	gb_pvfc.o.order_id	1	100.00	
2	gb_pvfc.op.finish_id	1	100.00	
4	gb_pvfc.op.prod_id	1	100.00	

5.3 SQL QUERY 3 (Table Join 3)

Business question:For each country and product, show total sales amount.

SQL:

```

SELECT
co.country_name,
p.prod_id,
p.name AS prod_name,
SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) AS total_sales
FROM gb_country co
JOIN gb_cust c ON co.country_id = c.country_id

```

```

JOIN gb_order o ON c.cust_id = o.cust_id
JOIN gb_ord_prod op ON o.order_id = op.order_id
JOIN gb_prod p ON op.prod_id = p.prod_id
GROUP BY
co.country_name,
p.prod_id,
p.name
ORDER BY
co.country_name,
total_sales DESC;

```

SCREENSHOT

```

100 SELECT
101 co.country_name,
102 p.prod_id,
103 p.name AS prod_name,
104 SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) AS total_sales
105 FROM gb_country co
106 JOIN gb_cust c ON co.country_id = c.country_id
107 JOIN gb_order o ON c.cust_id = o.cust_id
108 JOIN gb_ord_prod op ON o.order_id = op.order_id
109 JOIN gb_prod p ON op.prod_id = p.prod_id
110 GROUP BY
111 co.country_name,
112 p.prod_id,
113 p.name
114 ORDER BY
115 co.country_name,
116 total_sales DESC;
117

```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	co		ALL	PRIMARY				10	100.00	Using temporary; Using filesort
1	SIMPLE	c		ref	PRIMARY, idx_gb_cust_country...	idx_gb_cust_country_state	1	gb_pvfco.country_id	8	100.00	Using index
1	SIMPLE	o		ref	PRIMARY, idx_gb_order_cust_id	idx_gb_order_cust_id	4	gb_pvfco.cust_id	1	100.00	Using index
1	SIMPLE	op		ref	PRIMARY, gb_ord_prod_gb_pr...	PRIMARY	8	gb_pvfco.order_id	1	100.00	
1	SIMPLE	p		eq_ref	PRIMARY	PRIMARY	4	gb_pvfco.op.prod_id	1	100.00	

STATISTICS 13 Result 14 Explain Explain

Action Output Time Action Response

91 01:00:37 SELECT p.prod_id, p.name AS product_name, c.country_name, SUM(op.quant) AS total_quantity FROM gb_prod p JOIN gb_ord_prod op ON p.prod_id = op.prod_id JOIN gb_order o ON op.order_id = o.order_id JOIN gb_cust cu ON o.cu... 33 row(s) returned

id	select_type	table	partitions	type	possible_keys	key	key_len	ref
1	SIMPLE	co		ALL	PRIMARY			
1	SIMPLE	c		ref	PRIMARY, gb_cust_gb_country_fk	gb_cust_gb_country_fk	1	gb_pvfco.country_id
1	SIMPLE	o		ref	PRIMARY, idx_gb_order_cust_id	idx_gb_order_cust_id	4	gb_pvfco.cust_id
1	SIMPLE	op		ref	PRIMARY, gb_ord_prod_gb_prod_fk	PRIMARY	8	gb_pvfco.order_id
1	SIMPLE	p		eq_ref	PRIMARY	PRIMARY	4	gb_pvfco.op.prod_id

key...	ref	rows	filtered	Extra
		10	100.00	Using temporary; Using filesort
1	gb_pvfco.country_id	8	100.00	Using index
4	gb_pvfco.cust_id	1	100.00	Using index
8	gb_pvfco.order_id	1	100.00	
4	gb_pvfco.op.prod_id	1	100.00	

5.4 SQL QUERY 4 (Subqueries 1)

Business question:

PVFC wants to identify which customers have purchased furniture products priced

above the company's average unit price. In order to determine which customer segments show greater interest in high-value products.

SQL:

```
SELECT cu.cust_id,
cu.fname,
cu.lname,
c.country_name
FROM gb_cust cu JOIN gb_country c ON c.country_id = cu.country_id
WHERE cu.cust_id IN (
SELECT DISTINCT o.cust_id
FROM gb_order o
JOIN gb_ord_prod op ON o.order_id = op.order_id
JOIN gb_prod p ON op.prod_id = p.prod_id
WHERE p.unit_price >
(SELECT AVG(unit_price) FROM gb_prod));
```

SCREENSHOT

The screenshot displays a SQL query and its execution plan. The query is as follows:

```
SELECT cu.cust_id,
cu.fname,
cu.lname,
c.country_name
FROM gb_cust cu JOIN gb_country c ON c.country_id = cu.country_id
WHERE cu.cust_id IN (
SELECT DISTINCT o.cust_id
FROM gb_order o
JOIN gb_ord_prod op ON o.order_id = op.order_id
JOIN gb_prod p ON op.prod_id = p.prod_id
WHERE p.unit_price >
(SELECT AVG(unit_price) FROM gb_prod));
```

The execution plan is shown below:

ID	SELECT_TYPE	TABLE	PARTITIONS	TYPE	POSSIBLE_KEYS	KEY	KEY_LEN	REF	COST	ESTIMATED_ROWS	EXTRA
1	PRIMARY	c	ALL	PRIMARY			10		100.00		
1	PRIMARY	cu		ref	PRIMARY,gb_cust_gb_country_fk	gb_cust_gb_country_fk	8	1	100.00		
1	PRIMARY	<subquery2>		eq_ref	<auto_distinct_key>	<auto_distinct_key>	5	1	100.00		
2	MATERIALIZED	p	ALL	PRIMARY			25		23.23		Using where
2	MATERIALIZED	op		ref	PRIMARY,gb_ord_prod_gb_prod_fk	gb_ord_prod_gb_prod_fk	12	1	100.00		Using index
2	MATERIALIZED	o		eq_ref	PRIMARY,idx_gb_order_cust_id	PRIMARY	1	1	100.00		
3	SUBQUERY	gb_prod	ALL				25		100.00		

The execution plan also includes a 'Tabular Explain' section with the following details:

ID	SELECT_TYPE	TABLE	PARTITIONS	TYPE	POSSIBLE_KEYS	KEY	KEY_LEN	KEY...
1	PRIMARY	c		ALL	PRIMARY			
1	PRIMARY	cu		ref	PRIMARY,gb_cust_gb_country_fk	gb_cust_gb_country_fk	8	1
1	PRIMARY	<subquery2>		eq_ref	<auto_distinct_key>	<auto_distinct_key>	5	4
2	MATERIALIZED	p		ALL	PRIMARY			
2	MATERIALIZED	op		ref	PRIMARY,gb_ord_prod_gb_prod_fk	gb_ord_prod_gb_prod_fk	12	4
2	MATERIALIZED	o		eq_ref	PRIMARY,idx_gb_order_cust_id	PRIMARY	1	8
3	SUBQUERY	gb_prod		ALL				

key...	ref	rows	filtered	Extra
		10	100.00	
1	gb_pvfc.c.country_id	8	100.00	
4	gb_pvfc.cu.cust_id	1	100.00	
		25	33.33	Using where
4	gb_pvfc.p.prod_id	12	100.00	Using index
8	gb_pvfc.op.order_id	1	100.00	
		25	100.00	

5.5 SQL QUERY 5 (Subqueries 2)

Business question: Find customers whose total spending is above the average total spending of all customers.

SQL:

```

SELECT
c.cust_id,
c.fname,
c.lname,
SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) AS customer_total
FROM gb_cust c
JOIN gb_order o ON c.cust_id = o.cust_id
JOIN gb_ord_prod op ON o.order_id = op.order_id
JOIN gb_prod p ON op.prod_id = p.prod_id
GROUP BY
c.cust_id,
c.fname,
c.lname
HAVING
SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) >
(
SELECT AVG(customer_total)
FROM (
SELECT
c2.cust_id,
SUM(op2.quant * p2.unit_price * (1 - op2.discount_pct/100)) AS
customer_total
FROM gb_cust c2
JOIN gb_order o2 ON c2.cust_id = o2.cust_id
JOIN gb_ord_prod op2 ON o2.order_id = op2.order_id
JOIN gb_prod p2 ON op2.prod_id = p2.prod_id
GROUP BY
c2.cust_id

```

) AS t
);

SCREENSHOT

124 +
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

Tabular Explain

id	select_type	table	partitions	type	possible_keys	key	key...
1	PRIMARY	c		ALL	PRIMARY		
1	PRIMARY	c		ref	PRIMARY/idx_gb_order_cust_id	idx_gb_order_cust_id	4
1	PRIMARY	op		ref	PRIMARY,gb_ord_prod_gb_prod_fk	PRIMARY	8
1	PRIMARY	p		eq_ref	PRIMARY	PRIMARY	4
2	SUBQUERY	<derived3>		ALL			
3	DERIVED	c2		index	PRIMARY,gb_cust_gb_country_fk,...	PRIMARY	4
3	DERIVED	o2		ref	PRIMARY/idx_gb_order_cust_id	idx_gb_order_cust_id	4
3	DERIVED	op2		ref	PRIMARY,gb_ord_prod_gb_prod_fk	PRIMARY	8
3	DERIVED	p2		eq_ref	PRIMARY	PRIMARY	4

key... ref rows filtered Extra

		83	100.00	Using temporary
4	gb_pvfc.c.cust_id	1	100.00	Using index
8	gb_pvfc.o.order_id	1	100.00	
4	gb_pvfc.op.prod_id	1	100.00	
		303	100.00	
4		83	100.00	Using index
4	gb_pvfc.c2.cust_id	1	100.00	Using index
8	gb_pvfc.o2.order_id	1	100.00	
4	gb_pvfc.op2.prod_id	1	100.00	

5.6 SQL QUERY 6 (Subqueries 3)

Business question: Which customers have individual orders that exceed their own average order value, and what does this tell us about their purchasing behavior?

SQL:

```
SELECT
o.order_id,
o.cust_id,
SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) AS order_total
FROM gb_order o
JOIN gb_ord_prod op ON o.order_id = op.order_id
JOIN gb_prod p ON op.prod_id = p.prod_id
GROUP BY
o.order_id,
o.cust_id
HAVING
SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) >
(
SELECT AVG(inner_order_total)
FROM (
SELECT
o2.order_id,
SUM(op2.quant * p2.unit_price * (1 - op2.discount_pct/100)) AS
inner_order_total
FROM gb_order o2
JOIN gb_ord_prod op2 ON o2.order_id = op2.order_id
JOIN gb_prod p2 ON op2.prod_id = p2.prod_id
WHERE o2.cust_id = o.cust_id
GROUP BY
o2.order_id
) AS t
);
```

SCREENSHOT

```

166 SELECT
167 o.order_id,
168 o.cust_id,
169 SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) AS order_total
170 FROM gb_order o
171 JOIN gb_ord_prod op ON o.order_id = op.order_id
172 JOIN gb_prod p ON op.prod_id = p.prod_id
173 GROUP BY
174 o.order_id,
175 o.cust_id
176 HAVING
177 SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) >
178 {
179 SELECT AVG(inner_order_total)
180 FROM {
181 SELECT
182 o2.order_id,
183 SUM(op2.quant * p2.unit_price * (1 - op2.discount_pct/100)) AS
184 inner_order_total
185 FROM gb_order o2
186 JOIN gb_ord_prod op2 ON o2.order_id = op2.order_id
187 JOIN gb_prod p2 ON op2.prod_id = p2.prod_id
188 WHERE o2.cust_id = o.cust_id
189 GROUP BY
190 o2.order_id
191 } AS t
192 }
193
194

```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	p		ALL	PRIMARY				25	100.00	Using temporary
1	PRIMARY	op		ref	PRIMARY,gb_ord_prod_gb_pr...	gb_ord_prod_gb_prod_fk	4	gb_pvf.c.p.prod_id	12	100.00	
1	PRIMARY	o		eq_ref	PRIMARY,idx_gb_order_cust_id	PRIMARY	8	gb_pvf.c.op.order_id	1	100.00	
2	DEPENDENT SUBQU...	<derived>		ALL					3	100.00	
3	DEPENDENT DE...	o2		ref	PRIMARY,idx_gb_order_cust_id	idx_gb_order_cust_id	4	func	1	100.00	Using index; Using temporary
3	DEPENDENT DE...	op2		ref	PRIMARY,gb_ord_prod_gb_pr...	PRIMARY	8	gb_pvf.c.o2.order_id	1	100.00	
3	DEPENDENT DE...	p2		eq_ref	PRIMARY	PRIMARY	4	gb_pvf.c.op2.prod_id	1	100.00	

STATISTICS 13 Result 14 Explain Explain Explain Explain Explain

Action Output 0

Time	Action	Response
96 01:18:50	EXPLAIN SELECT o.order_id, o.cust_id, SUM(op.quant * p.unit_price * (1 - op.discount_pct/100)) AS order_total FROM gb_order o JOIN gb_ord_prod op ON o.order_id = op.order_id JOIN gb_prod p ON op.prod_id = p.prod_id GROUP BY...	OK

Tabular Explain							
id	select_type	table	partitions	type	possible_keys	key	key...
1	PRIMARY	p		ALL	PRIMARY		
1	PRIMARY	op		ref	PRIMARY,gb_ord_prod_gb_prod_fk	gb_ord_prod_gb_prod_fk	4
1	PRIMARY	o		eq_ref	PRIMARY,idx_gb_order_cust_id	PRIMARY	8
2	DEPENDENT SUBQU...	<derived>		ALL			
3	DEPENDENT DERIVED	o2		ref	PRIMARY,idx_gb_order_cust_id	idx_gb_order_cust_id	4
3	DEPENDENT DERIVED	op2		ref	PRIMARY,gb_ord_prod_gb_prod_fk	PRIMARY	8
3	DEPENDENT DERIVED	p2		eq_ref	PRIMARY	PRIMARY	4

key...	ref	rows	filtered	Extra
		25	100.00	Using temporary
4	gb_pvf.c.p.prod_id	12	100.00	
8	gb_pvf.c.op.order_id	1	100.00	
		3	100.00	
4	func	1	100.00	Using index; Using temporary
8	gb_pvf.c.o2.order_id	1	100.00	
4	gb_pvf.c.op2.prod_id	1	100.00	

5.7 SQL QUERY 7 (ROLLUP)

Business question:

For each customer, how many items have they ordered in total, and what is the overall total quantity ordered by all customers?

SQL:

```

SELECT
c.cust_id,
CONCAT(c.fname, ' ', c.lname) AS customer_name,

```

```

SUM(op.quant) AS total_quantity
FROM gb_cust c
JOIN gb_order o ON c.cust_id = o.cust_id
JOIN gb_ord_prod op ON o.order_id = op.order_id
GROUP BY c.cust_id, c.fname, c.lname WITH ROLLUP;

```

SCREENSHOT

Tabular Explain

id	select_type	table	partitions	type	possible_keys	key	key...	rows	filtered	Extra
1	SIMPLE	c		ALL	PRIMARY			83	100.00	Using filesort
1	SIMPLE	o		ref	PRIMARY,idx_gb_order_cust_id	idx_gb_order_cust_id	4	1	100.00	Using index
1	SIMPLE	op		ref	PRIMARY,idx_gb_order_prod_id	PRIMARY	8	1	100.00	

key...	ref	rows	filtered	Extra
4	gb_pvfc.c.cust_id	1	100.00	Using index
8	gb_pvfc.o.order_id	1	100.00	

5.8 SQL QUERY 8 (RANK)

Business question:

Which products are ordered the most, and what is the ranking of each product based on total quantity ordered?

SQL:

```

SELECT b.prod_id, b.name AS prod_name, p.total_quantity,
RANK() OVER (ORDER BY p.total_quantity DESC) AS quantity_rank
FROM gb_prod b,( SELECT prod_id, SUM(quant) AS total_quantity
FROM gb_ord_prod
GROUP BY prod_id) p
WHERE b.prod_id = p.prod_id
ORDER BY quantity_rank;

```

SCREENSHOT

```

207 SELECT b.prod_id, b.name AS prod_name, p.total_quantity,
208 RANK() OVER (ORDER BY p.total_quantity DESC) AS quantity_rank
209 FROM gb_prod b,( SELECT prod_id, SUM(quant) AS total_quantity
210 FROM gb_ord_prod
211 GROUP BY prod_id) p
212 WHERE b.prod_id = p.prod_id
213 ORDER BY quantity_rank;
214

```

1206

Tabular Explain

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived2>		ALL					300	100.00	Using temporary; Using filesort
1	PRIMARY	b		eq_ref	PRIMARY	PRIMARY	4	p.prod_id	1	100.00	
2	DERIVED	gb_ord_prod		index	PRIMARY,gb_ord_prod_gb_fin...	gb_ord_prod_gb_prod_fk	4		300	100.00	

STATISTICS 13

Result 14

Explain

Explain

Explain

Explain

Explain

Explain

Explain

Action Output

Time

Action

Response

98

01:22:38

EXPLAIN SELECT b.prod_id, b.name AS prod_name, p.total_quantity, RANK() OVER (ORDER BY p.total_quantity DESC) AS quantity_rank FROM gb_prod b,(SELECT prod_id, SUM(quant) AS total_quantity FROM gb_ord_prod GROUP BY pr...

OK

Tabular Explain

id	select_type	table	partitions	type	possible_keys	key	key...
1	PRIMARY	<derived2>		ALL			
1	PRIMARY	b		eq_ref	PRIMARY	PRIMARY	4
2	DERIVED	gb_ord_prod		index	PRIMARY,gb_ord_prod_gb_finish_fk...	gb_ord_prod_gb_prod_fk	4

key...	ref	rows	filtered	Extra
		300	100.00	Using temporary; Using filesort
4	p.prod_id	1	100.00	
4		300	100.00	

6. Conclusion

This project was a great opportunity for our team to work through the entire design, build and analytic processes of a relational database. The process of creating logical and relational models, importing and validating data into the tables and creating a wide range of SQL query types from basic joins to complex subqueries, analytic functions and performance improvement indexing, combined with the knowledge gained about how structured data supports the operations and analytics of a business environment directly shows how much value there is in having high-quality database models and schemas, and how these lead to producing accurate results and more efficient queries.

Through this experience, we learned that the quality of a data model and a schema is essential to obtaining accurate results and having efficient queries. It was also shown to us that the importance of naming conventions in the development of models, understanding the relationships between the foreign and primary keys in a model, and developing good SQL coding practices all help to prevent errors and correct incomplete data within a database. We also learned about the impact of using indexes and how they can be an important part of improving performance while working with large data sets.

In conclusion, this project helped us to obtain a better understanding of how data can be leveraged to help answer some of the fundamental questions facing businesses, such as identifying and targeting high-value customers, determining and managing the demand for a product and how to assess sales performances across multiple nations. The comprehensive focus of this project provides a framework for our future work as it relates to working with data, including data analytics, business intelligence, database management, and then developing a model-driven analytics approach to problem solving to support our broader business analytics initiatives.

7. Team reflection

In this project, our team came across several problems. At the beginning of the data import process, we noticed that some file names didn't match the tables specified in the project instructions. For instance, there was a file named *ga_doc_dis_pat_stage*, while the table we needed was *gb_ord_prod_stage*. After reviewing the file contents, we confirmed that it was the correct data file but had been named incorrectly, so we renamed it before importing. In addition, some table names in the instructions didn't match the actual table names in the database. For instance, the instructions referred to *gb_countries*, whereas our database contained *gb_country*. As a result, some queries failed to run until the naming issues were corrected. This experience illustrates the importance of carefully verifying naming accuracy before executing SQL queries.

We also faced several technical difficulties during query development. In some cases, team members confused the use of **WHERE** and **HAVING**, particularly when working with aggregate functions. Additionally, when writing multi-table JOIN queries, we occasionally forgot to include table prefixes or aliases, which led to syntax errors. To address this issue, we adopted the practice of defining all table aliases at the beginning of each query, which significantly reduced errors and improved query clarity.

From our perspective, there were moments when multiple members unknowingly worked on the same section, while other sections were temporarily unassigned. In some cases, minor mistakes introduced during editing weren't immediately identified. To improve coordination and consistency, we assigned one team member to review the final document, ensuring that the structure, formatting, and content were clear and consistent throughout the report.

8. Individual reflection

Rui Liu

The PVFC Database project has truly helped me learn how to use SQL to analyze real-world business problems. From the initial design of the logical and relational models to subsequent tasks such as data insertion, data dictionary queries, table joins, subqueries, analytical functions, as well as creating indexes and examining execution plans. The entire process deepened my understanding of the relationship between database structure and query performance. It also enabled me to better integrate theoretical knowledge from coursework with practical business scenarios.

From a career development perspective, this SQL project taught me not only the technical skills of writing queries but, more importantly, how to use data to answer genuine business questions. This competency closely align with the practical data analysis tasks performed in real-world enterprises.

Clark Zhang

As a beginner in database management, this project was a big learning curve for me. The most challenging part wasn't just the SQL code, but coordinating with my team. Since some members were traveling internationally or very busy, we had significant time differences and couldn't schedule Zoom meetings easily. It felt like we were working in different worlds.

This situation taught me a very practical lesson: don't wait for instant replies. In the beginning, I would ask simple questions and wait. But because of the time gap, this wasted a lot of time. I learned that I had to be very specific when I posted a message. Instead of just saying "I have an error," I learned to post the code, the error screenshot, and what I tried to fix, all at once. This way, when my teammates woke up or landed, they could solve the problem immediately without asking me for more details.

Technically, the new tasks (Task 6 and 7) were very helpful. Before this, I only cared if my code ran without errors. But seeing the Execution Plan graphs showed me "under the hood" of the database. It made me realize that writing code is easy, but writing code that the database likes is harder.

Ya cheng Hsieh

The PVFC Database Project provided me with a deep and practical understanding of the principles of database management, SQL, and analytical thinking. In particular, it gave me confidence in understanding the full database lifecycle such as data modelling, data importation, using multiple table joins, subqueries, analytical functions, and indexes.

Through this experience, I enhanced my skills significantly. For example, when I found out my table names did not match either the stage files I had or the query instructions, I learned to investigate further to find out where the problem originated, rather than assuming there was an error in my SQL. I also learned the importance of foreign keys, database constraints, and proper join command legibility, that just one small mistake in my SQL can have potentially devastating consequences for the final results.

Overall, this database project greatly improved my technical capabilities, problem-solving abilities, and confidence in my ability to use SQL for business decision-making purposes. I know these abilities will be importance to me in my future data analyst and business analyst work in the finance and business intelligence areas of the economy.

Kanokkan Kanjanpiban

This project helped me connect database design concepts with real SQL implementation. One of the most important things I learned was the significance of managing foreign key relationships correctly during data insertion. If tables are populated in the wrong order or if data types don't match, foreign key constraints can prevent the data from loading successfully. This made me more aware of how database structure directly affects data integrity.

Moreover, I learned the importance of being **precise** when writing SQL queries. Small details, such as table names, column names, and aliases, can easily cause errors if not handled carefully. Through repeated practice with joins, subqueries, and analytical functions, I became more confident in writing complex queries and understanding how different tables interact within a relational database.

Overall, this project strengthened my understanding of how well-designed databases support accurate analysis and efficient query performance. The skills I developed during this project will be valuable for my future academic work and for potential roles in data analytics or business analytics, where working with structured data and writing reliable SQL queries are essential.